Project Report

On

CHAKRAVYUH - AN ADVANCED WEB SECURITY AWARENESS PORTAL

Learning Cybersecurity Through Interactive Games & Simulations

Submitted by

Yash Jadhav [Enrolment no. - 2405101200015]

Ananya Yadav [Enrolment no. - 2505101279004]

Jaivin Vachhani [Enrolment no. - 2405101200043]

as

Partial fulfilment of Semester V

of BCA Hons & B.Sc. IT Hons

for A.Y. 2026-2027

Under the Guidance of

Prof. Chaitali Nayka

Submitted To

Parul Institute of Computer Application,

Faculty of IT & Computer Science

Parul University

Acknowledgement

The successful completion of this project required guidance, assistance, and support throughout the development process. We are grateful to Parul Institute of Computer Applications and the Faculty of IT & Computer Science for providing the opportunity and resources to undertake this project. We express our sincere gratitude to our project guide and faculty members for their guidance, feedback, and encouragement. We also thank our parents, classmates, and all those who supported us during the planning, development, testing, and documentation of the Chakravyuh - An Advanced Web Security Awareness Portal.

I respect and thank DR. PRIYA SWAMINARAYAN for providing us an opportunity to do the project work and giving us all support and guidance which made us complete the project duly. We are extremely thankful to Mam for providing her support and guidance, although she had a busy schedule managing the academic affairs.

We would not forget to remember DR. PRIYA SWAMINARAYAN for her encouragement and more over for her timely support and guidance till the completion of our project work.

We are also thankful to Dr. Hina Chokshi, Vice-Principal, for her encouragement, support, and guidance during the completion of our project work.

We also express our sincere gratitude to Prof. Vipul Gamit, HOD, for his valuable support and guidance throughout the project and for encouraging us during the completion of this work.

We owe our deep gratitude to our project guide Prof. CHAITALI NAYKA who took keen interest on our project work and guided us all along, till the completion of our project work by providing all the necessary information for developing a good system.

Yash Jadhav [Enrolment no. - 2405101200015]

Ananya Yadav [Enrolment no. - 2505101279004]

Jaivin Vachhani [Enrolment no. - 2405101200043]



PARUL INSTITUTE OF COMPUTER APPLICATION

CERTIFICATE

This is to certify that Yash Jadhav, Ananya Yadav, and Jaivin Vachhani, the student(s) of Parul Institute of Computer Applications, have satisfactorily completed the project entitled “Chakravyuh - An Advanced Web Security Awareness Portal” as a part of course curriculum in BCA (Hons.) Semester V for the academic year 2026-2027 under the guidance of Prof. Chaitali Nayka.

Enrolment Number: 2405101200015

Enrolment Number: 2505101279004

Enrolment Number: 2405101200043

Quality of work	Grade	Sign of Internal guide
Poor / Average / Good / Excellent	B / B+ / A / A+	

Date of submission: _____

HOD,
Prof. Vipul Gamit

Vice-Principal,
Dr. Hina Chokshi

Dean,
Dr. Priya Swaminarayan

TABLE OF CONTENTS

Sr. No.	Topic	Page No.
1	**Introduction to Project System**	**1**
2	**System Requirement Specification**	**11**
2.1	Introduction to SRS	11
2.2	Hardware Requirement	12
2.3	Software Requirement	12
2.4	System Users	13
2.5	Description of User Role	13
2.6	System Modules	14
2.7	Description of Modules	14
2.8	Timeline Chart	17
3	**System Flow Diagram**	**23**
4	**Data Flow Diagram (All Levels of DFDs)**	**26**
5	**Use Case Diagram**	**30**
6	**Data Dictionary**	**36**
7	**Screenshots of Development Phase - 1**	**40**
8	**Screenshots of Development Phase - 2**	**42**
9	**Screenshots of Development Phase - 3**	**45**
10	**Conclusion**	**48**
11	**Future Enhancement**	**50**
12	**References**	**51**

ABSTRACT

Chakravyuh - An Advanced Web Security Awareness Portal is a Python (Flask)-based web application that teaches college students to recognise and avoid common cybersecurity threats through practical, gamified learning. It addresses the gap between knowing security rules and applying them in realistic situations. The portal provides four interactive modules: Phishing Spotter, Password Strength Lab, Social Engineering Simulator, and Quiz & Challenge. A context-scoped Google Gemini API assistant explains answers within the current module, while HaveIBeenPwned supports password-breach checking through the k-Anonymity approach. Progress tracking, scoring, a leaderboard, and an administrative content workflow support continued learning and reuse by the college. The backend uses Flask, SQLite, and SQLAlchemy with a browser-based interface. The semester-scoped system also demonstrates authentication, databases, APIs, cybersecurity awareness exercises, gamification, and administration, while allowing core functionality to remain usable when external services are unavailable.

1. Introduction to Project System

The Chakravyuh - An Advanced Web Security Awareness Portal is a web-based interactive platform designed to educate users about common cybersecurity threats through practical, gamified learning.

The project focuses on reducing the gap between knowing cybersecurity rules and being able to recognize threats in real-world situations. The system uses interactive modules, scoring, feedback, and contextual AI explanations to make cybersecurity awareness more engaging and understandable.

1.1 Project Rationale

Comparison of new system with existing system

Existing systems (TryHackMe, KnowBe4, Google's Phishing Quiz) are mostly subscription-based or enterprise-oriented, built for corporate training rather than individual or student learners.

They test static knowledge through one-off quizzes and rarely explain why an answer was right or wrong in a contextual, conversational way.

None of them combine awareness modules with a real, personal breach lookup (HaveIBeenPwned) inside the same platform.

Our system is free, browser-based, and built on a lightweight Python/Flask stack the college can host and extend for future batches at no licensing cost.

A scoped AI assistant (Gemini API) gives plain-English, per-question explanations instead of generic answer keys.

Progress dashboard, leaderboard, and admin content panel make it usable as an ongoing, growing institutional tool rather than a one-time exercise.

1.2 Research and Comparison of Existing Systems

In today's digital age, cybersecurity threats are growing at an alarming rate. Phishing attacks, weak passwords, social engineering, and data breaches have become everyday realities - yet the average internet user, especially students, remains dangerously underprepared. The need for accessible, engaging, and effective cybersecurity awareness training has never been greater.

Before designing and developing the Chakravyuh - An Advanced Web Security Awareness Portal, it is essential to study and analyse the systems that already exist in this space. Researching existing platforms helps the team understand what has already been done, what works well, what gaps remain, and how the proposed system can add genuine value rather than simply duplicating what is already available.

This document presents a structured research and comparative analysis of five widely recognised existing cybersecurity awareness systems. Each system is evaluated on its features, target audience, strengths, and weaknesses. The analysis concludes with a direct comparison between existing systems and the proposed portal, highlighting the advantages and limitations of the proposed solution.

The following methods were used to research and evaluate existing systems:

- Direct usage and exploration - The team accessed and interacted with freely available platforms (Google Phishing Quiz, OWASP WebGoat) to evaluate the user experience firsthand.
- Official documentation and websites - Feature lists, pricing pages, and product descriptions from official sources were reviewed for each platform.
- Academic and industry literature - Research papers on game-based learning, cybersecurity awareness training effectiveness, and e-learning platforms were consulted.
- OWASP and NIST publications - Industry frameworks were referenced to understand the standard expectations of a cybersecurity awareness tool.

The following five systems were selected for analysis based on their relevance to cybersecurity awareness, their popularity, and their diversity in approach - ranging from simple quizzes to full enterprise training platforms.

3.1 Google Phishing Quiz - A free, browser-based quiz developed by Google in partnership with Jigsaw. It presents users with simulated email scenarios and asks them to identify whether each is a phishing attempt or a legitimate email.

3.2 KnowBe4 - One of the world's largest security awareness training platforms, used primarily by enterprises and organisations. It offers simulated phishing campaigns, training videos, assessments, and detailed reporting dashboards.

3.3 Cybrary - An online cybersecurity learning platform offering courses, labs, and career paths for IT professionals and students. It covers a wide range of topics from ethical hacking to compliance training.

3.4 PhishTank - A community-driven platform where users can submit, verify, and track phishing URLs. It is primarily a threat intelligence resource rather than a training tool, but it contributes to phishing awareness.

3.5 OWASP WebGoat - A deliberately insecure web application developed by the OWASP Foundation. It is designed for developers and security professionals to learn about web vulnerabilities by exploiting them in a safe, controlled environment.

1.3 Individual System Analysis

1.3.1 Google Phishing Quiz

Parameter	Details
Developer / Owner	Google / Jigsaw
Purpose	Teach users to identify phishing emails and messages
Target Audience	General public, non-technical users
Key Features	Email scenario simulation, instant feedback, explanation of phishing indicators
Technology Used	Web-based (browser), no account required
Accessibility	Freely accessible at phishingquiz.withgoogle.com
Cost	Free
Limitations	Single game type only; no progress tracking; no user accounts; no scoring system; content cannot be updated by users

1.3.2 KnowBe4

Parameter	Details
Developer / Owner	KnowBe4 Inc.
Purpose	Enterprise-level security awareness training and phishing simulation
Target Audience	Corporate employees, IT departments, enterprise organisations
Key Features	Simulated phishing campaigns, training video library, risk scoring, admin dashboard, reporting and analytics, compliance tracking
Technology Used	Cloud-based SaaS platform
Accessibility	Subscription-based; requires organisational sign-up
Cost	Paid (starts at approximately $30 per user per year)
Limitations	Expensive for individuals or small teams; not designed for students; requires organisational admin setup; no game-based learning; content is corporate-focused

1.3.3 Cybrary

Parameter	Details
Developer / Owner	Cybrary Inc.

Parameter	Details
Purpose	Online cybersecurity education and career development
Target Audience	IT professionals, cybersecurity students, career changers
Key Features	Video courses, virtual labs, assessments, career paths, certifications, progress tracking
Technology Used	Cloud-based web platform
Accessibility	Free tier available; premium features require paid subscription
Cost	Free (limited) / Paid plans from approximately $99/year
Limitations	Heavily text and video based; minimal gamification; designed for intermediate to advanced learners; not beginner-friendly for non-technical users; paid wall for key features

1.3.4 PhishTank

Parameter	Details
Developer / Owner	Cisco (originally OpenDNS)
Purpose	Community-driven phishing URL verification and tracking database
Target Audience	Security researchers, developers, IT professionals
Key Features	Phishing URL submissions, community voting, API access for developers, phishing statistics
Technology Used	Web-based platform with public API
Accessibility	Free to use and access
Cost	Free
Limitations	Not a training or awareness tool; purely technical and data-oriented; no game elements; no learning feedback; completely unsuitable for non-technical or beginner users

1.3.5 OWASP WebGoat

Parameter	Details
Developer / Owner	OWASP Foundation
Purpose	Hands-on learning of web application vulnerabilities by exploiting a deliberately insecure app

Parameter	Details
Target Audience	Developers, security professionals, advanced students
Key Features	Interactive vulnerability lessons, code injection labs, SQL injection practice, broken auth exercises, detailed lesson explanations
Technology Used	Java-based web application (self-hosted)
Accessibility	Free and open source; requires local installation
Cost	Free
Limitations	Requires technical setup and installation; extremely complex for beginners; not suitable for general public or non-technical users; focuses on exploitation rather than awareness; no gamification or scoring

1.4 Advantages & Disadvantages of Existing Systems

1.5.1 Google Phishing Quiz

Advantages:

- Completely free with no registration required, making it immediately accessible to anyone
- Clean, simple interface that any user can understand within seconds
- Realistic email scenarios based on actual phishing tactics (brand impersonation, urgency tactics, fake login pages)
- Instant feedback after each answer with a clear explanation of why the email was or was not phishing
- Backed by Google's credibility, which lends trust to the content
- Available in multiple languages, making it globally accessible

Disadvantages:

- Only one type of activity - identifying phishing emails - with no variety of modules
- No user accounts, so there is no way to track progress over time or return to previous sessions
- No scoring system or competitive element to motivate continued use
- Content is static and cannot be updated or customised by educators or institutions
- Covers only phishing - does not address password hygiene, social engineering, malware awareness, or safe browsing
- No admin panel or reporting - educators cannot see how their students performed
- After completing the quiz once, there is little reason for a user to return

1.5.2 KnowBe4

Advantages:

- Extremely comprehensive platform with thousands of training assets including videos, quizzes, and simulated phishing campaigns
- Detailed admin dashboard with analytics, risk scores, and compliance tracking
- Simulated phishing campaigns allow admins to send fake phishing emails to employees and track who clicked
- Regularly updated content library reflecting the latest real-world threats
- Used and trusted by thousands of enterprise organisations globally
- Supports integration with HR systems and identity providers like Active Directory

Disadvantages:

- Expensive - pricing is enterprise-focused and completely out of reach for individual students or small college teams
- Not designed for students or non-technical users; content assumes a professional working context
- Heavily video and text based with minimal interactivity or game elements
- Requires significant admin setup and ongoing management - not suitable for a simple demo environment
- No publicly accessible version; everything requires an organisational account
- The corporate tone and jargon make it inaccessible and unengaging for a student demographic

1.5.3 Cybrary

Advantages:

- Wide range of cybersecurity topics from beginner to advanced level
- Structured career paths help users understand what to learn in what order
- Virtual labs allow hands-on practice in controlled environments
- Free tier provides access to a meaningful amount of content
- Community features including forums and discussion boards
- Recognised certifications and skill assessments

Disadvantages:

- The free tier is increasingly limited; most valuable content is behind a paid subscription
- Almost entirely video and article based - very little interactive or game-based content
- Interface is dense and can be overwhelming for a complete beginner who just wants a quick awareness lesson
- Primarily oriented toward people pursuing cybersecurity as a career, not casual users seeking basic awareness
- No gamification elements like leaderboards, badges, or competitive scoring to motivate non-technical learners
- Not customisable by institutions - an educator cannot add their own content or scenarios

1.5.4 PhishTank

Advantages:

- Valuable and free resource for tracking real-world phishing URLs in near-real-time
- Community-driven model means the database is constantly updated with fresh submissions
- Developer-friendly API that can be integrated into other applications and security tools
- Transparent and open - anyone can verify, vote on, and discuss submissions
- Useful for researchers studying phishing patterns and trends

Disadvantages:

- Not a training or awareness tool in any meaningful sense for end users
- Completely unsuitable for non-technical or beginner users - requires understanding of URLs and phishing mechanics just to use
- No learning feedback, no explanations, no guidance - purely a data platform
- No game elements, no accounts for learners, no progress tracking
- The interface is minimal and uninviting for anyone not already in the security field
- Being a community-driven database means data quality can vary

1.5.5 OWASP WebGoat

Advantages:

- Completely free and open source - can be downloaded, run locally, and modified freely
- Excellent for learning about real web vulnerabilities through hands-on exploitation in a safe environment
- Covers a wide range of technical topics including SQL injection, XSS, broken authentication, and IDOR
- Lessons include detailed explanations and hints, making it self-guided for those with the right background
- Backed by the OWASP Foundation, the gold standard in web application security guidance
- Actively maintained with updates reflecting current OWASP Top 10

Disadvantages:

- Requires technical installation (Java, command line) - a significant barrier for non-technical users
- Designed exclusively for developers and security professionals; completely inappropriate for general students
- Focuses on exploitation and technical vulnerability understanding, not everyday user awareness
- No gamification, no leaderboard, no scoring - purely educational in a technical, task-based format
- Very intimidating interface that assumes prior knowledge of web application concepts
- Not suitable for raising awareness about phishing, password safety, or social engineering among general users

1.5 Proposed System Benefits

1.6.1 Cost and Accessibility

Every existing system that offers meaningful depth - KnowBe4, Cybrary's premium features - comes at a cost that students and college projects simply cannot afford. The proposed portal is built entirely on free and open-source technologies (Python, Flask, SQLite, Bootstrap) and can be hosted locally with zero ongoing cost. There are no subscription fees, no per-user charges, and no institutional contracts required.

1.6.2 Genuine Gamification

None of the existing systems reviewed offer a complete gamification experience. Google's quiz has no scoring or accounts. KnowBe4 and Cybrary have progress tracking but no leaderboards or competitive elements. The proposed system integrates scoring, a leaderboard, achievement badges, difficulty levels, and a user dashboard into a single platform - creating intrinsic motivation for continued engagement that existing tools lack.

1.6.3 Multiple Game Module Types in One Place

Each existing system is focused on a single approach - Google covers only phishing identification, WebGoat covers only technical exploitation, PhishTank is purely a database. The proposed system brings four distinct game types together: Phishing Spotter, Password Strength Lab, Social Engineering Simulator, and a Quiz & Challenge module. A student can experience variety within a single session rather than visiting multiple platforms.

1.6.4 Designed Specifically for Non-Technical Beginners

WebGoat and Cybrary are aimed at IT professionals and security students. KnowBe4 is aimed at corporate employees. None are designed with a first-year college student who has never heard of SQL injection in mind. The proposed portal's entire design philosophy - simple language, visual feedback, guided explanations after each answer - is built around making cybersecurity approachable for someone with no prior knowledge.

1.6.5 Customisable by the Institution

An educator or admin using the proposed system can log into the admin panel and add new phishing examples, quiz questions, or social engineering scenarios specific to their context - without touching any code. KnowBe4 offers this but only at enterprise pricing. No free system reviewed offers educator-controlled content management.

1.6.6 Local Deployment Capability

Most existing platforms are entirely cloud-dependent. The proposed system runs on a local Flask development server, meaning it works perfectly for a college demo or internal use without requiring

an internet connection. This is a practical advantage for institutions with unreliable internet or where data privacy concerns prevent cloud-hosted tools.

1.6.7 Relevance to the Target Demographic

Existing corporate tools use workplace scenarios - fake HR emails, expense report phishing, corporate password policies. These are meaningless to a 19-year-old student. The proposed system can use scenarios that actually resonate: fake college admission emails, WhatsApp scams, social media account takeovers, and OTP fraud - threats students genuinely encounter.

2. System Requirement Specification

Document Type	Final Project Report
Project Title	Chakravyuh - An Advanced Web Security Awareness Portal
Version	Final
Prepared For	Internal Guide / College Submission
Team Size	3 Members
Academic Year	2026-27
Project Type	Mini Project - Single Semester

2.1 Introduction to SRS

This System Requirement Specification (SRS) document defines the functional and non-functional requirements for the Chakravyuh - An Advanced Web Security Awareness Portal. It serves as the formal agreement between the project team and stakeholders on what the system will do, how it will perform, and what constraints it must operate within.

2.1.1 Scope

The Chakravyuh - An Advanced Web Security Awareness Portal is a web-based interactive platform that educates college students about common cybersecurity threats through gamified learning modules. The system includes a Phishing Spotter, Password Strength Lab, Social Engineering Simulator, and Quiz & Challenge module, supported by AI-generated explanations via the Google Gemini API and breach detection via the HaveIBeenPwned API.

2.1.2 Definitions & Abbreviations

Term	Definition
SRS	System Requirement Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
HIBP	HaveIBeenPwned - public breach database API
k-Anonymity	Privacy model where only a partial hash prefix is transmitted
ORM	Object Relational Mapper - SQLAlchemy in this project
bcrypt	Cryptographic hashing function used for password storage
Admin	Project team member with elevated portal management access

Term	Definition
User	Registered college student using the portal for learning
AI Explain	Feature powered by Gemini API that explains answers contextually

2.1.3 System Overview

The portal follows a client-server architecture built on Python Flask. Users interact with the system through a web browser. The Flask backend serves Jinja2-rendered HTML pages, handles authentication, manages game logic, communicates with the Gemini API for AI explanations, and proxies HaveIBeenPwned requests using the k-Anonymity model. All data is persisted in a SQLite database via SQLAlchemy ORM.

2.2 Hardware Requirement

The source project documentation does not prescribe fixed CPU, RAM, or storage minima. The documented deployment model requires a client device with a modern web browser and a local/server environment capable of running Python/Flask and SQLite. Internet access is required when Gemini and HaveIBeenPwned services are used; core game functionality is intended to remain usable locally when those external APIs are unavailable.

2.3 Software Requirement

Component	Chosen Technology	Justification
Backend Language	Python 3.x	Team familiarity; large support community; excellent web framework ecosystem
Web Framework	Flask	Lightweight, beginner-friendly, minimal boilerplate; ideal for semester scope
Frontend	HTML5, CSS3, JavaScript, Bootstrap 5	No additional learning curve; directly supports interactive game UI
Database	SQLite (dev) / MySQL-compatible (optional deployment)	SQLite requires zero setup for local demo; MySQL for optional deployment
ORM	SQLAlchemy	Abstracts database queries; reduces SQL injection risk; portable across DBs
Authentication	Flask-Login + bcrypt	Well-tested, standard session-based auth with hashed passwords

Component	Chosen Technology	Justification
AI Integration	Google Gemini API	Lightweight, cost-effective model for per-question AI explanations; API key stored server-side
Breach API	HaveIBeenPwned API v3	k-Anonymity model - only first 5 chars of SHA-1 hash sent; no full password transmitted
Version Control	Git + GitHub	Industry standard; free for students; enables team collaboration
Hosting (Demo)	Local dev server / Render.com (free tier)	Sufficient for college demo; no cost involved

2.4 System Users

User Type	Purpose	Main Access
Guest	First-time or unauthenticated visitor	Landing/about page; register; log in
Registered User	College learner using the awareness portal	Play modules; AI explanations; HIBP breach check; dashboard; leaderboard; badges
Admin	Privileged portal manager	Admin panel; content/question management; user management; score analytics

2.5 Description of User Role

Feature / Action	Guest (Not Logged In)	Registered User	Admin
View landing / about page	Yes	Yes	Yes
Register / Create account	Yes	-	-
Log in	Yes	Yes	Yes
Play game modules	No	Yes	Yes
Receive AI explanations (Gemini API)	No	Yes	Yes
Use breach checker (HIBP)	No	Yes	Yes
View own scores / dashboard	No	Yes	Yes

Feature / Action	Guest (Not Logged In)	Registered User	Admin
View leaderboard	No	Yes	Yes
Earn and view badges	No	Yes	Yes
Access admin panel	No	No	Yes
Add/Edit game content & questions	No	No	Yes
View all user scores	No	No	Yes

2.6 System Modules

Module	Description
1. Phishing Spotter	Classifies simulated emails as phishing or legitimate and provides immediate feedback.
2. Password Strength Lab	Checks password strength in real time and performs the documented HIBP breach check using k-Anonymity.
3. Social Engineering Simulator	Provides branching social-engineering scenarios for decision-based awareness practice.
4. Quiz & Challenge	Provides timed MCQ-based cybersecurity questions across documented difficulty levels.
Supporting services	Gamification, dashboard, leaderboard, admin content management, Gemini Explain This, and HIBP breach checking support the four core modules.

2.7 Description of Modules

2.7.1 User Account Management

ID	Requirement	Priority
FR-01.1	The system shall allow new users to register with a username, email, and password	Must Have
FR-01.2	The system shall authenticate users via username/email and password	Must Have
FR-01.3	The system shall store passwords as bcrypt hashes - never in plain text	Must Have
FR-01.4	The system shall maintain user sessions and support logout functionality	Must Have

ID	Requirement	Priority
FR-01.5	The system shall display a personalised dashboard showing progress and scores	Should Have

2.7.2 Game Modules

ID	Requirement	Priority
FR-02.1	The system shall provide a Phishing Spotter game where users classify emails as phishing or legitimate	Must Have
FR-02.2	The Phishing Spotter shall provide immediate feedback with an explanation after each answer	Must Have
FR-02.3	The system shall provide a Password Strength Lab with real-time strength indicator	Must Have
FR-02.4	The Password Lab shall display breach status via HaveIBeenPwned API (FR-06)	Must Have
FR-02.5	The system shall provide a Social Engineering Simulator with branching scenarios	Should Have
FR-02.6	The system shall provide a timed Quiz & Challenge module with MCQ questions	Must Have
FR-02.7	Quiz questions shall span Beginner, Intermediate, and Advanced difficulty	Should Have
FR-02.8	Each game module shall record the user score and timestamp to the database	Must Have

2.7.3 Gamification

ID	Requirement	Priority
FR-03.1	The system shall maintain a points-based scoring system across all game modules	Must Have
FR-03.2	The system shall display a public leaderboard showing top-scoring users	Should Have

ID	Requirement	Priority
FR-03.3	The system shall award achievement badges for completing modules and milestones	Could Have
FR-03.4	The user dashboard shall display total points, badges earned, and modules completed	Must Have

2.7.4 Admin Panel

ID	Requirement	Priority
FR-04.1	The system shall provide an admin panel accessible only to authorised admin users	Must Have
FR-04.2	Admin users shall be able to add, edit, and delete quiz questions and phishing examples	Must Have
FR-04.3	Admin panel shall display a summary of all registered users and their scores	Should Have
FR-04.4	Admin shall be able to reset user scores for retesting purposes	Could Have

2.7.5 AI-Powered Explanation

ID	Requirement	Priority
FR-05.1	After each answer, the system shall display an AI-generated explanation via Gemini API	Must Have
FR-05.2	Only question text and answer data shall be sent to Gemini - no PII transmitted	Must Have
FR-05.3	A loading indicator shall display while fetching; explanation loads within 5 seconds	Must Have
FR-05.4	If Gemini API fails, a static fallback explanation shall display - game shall not crash	Must Have
FR-05.5	Gemini API key shall be stored server-side as environment variable only	Must Have

ID	Requirement	Priority
FR-05.6	AI explanations shall not be persisted in the database - ephemeral by design	Must Have
FR-05.7	Explanation panel shall display an "AI Explanation" label badge	Should Have

2.7.6 Password Breach Checker

ID	Requirement	Priority
FR-06.1	Password Lab shall hash input with SHA-1, send only first 5 chars to HIBP API, compare locally	Must Have
FR-06.2	System shall display breach count or "No breaches found" message	Must Have
FR-06.3	Breach check shall trigger after 500ms debounce on input field change	Must Have
FR-06.4	If HIBP API is unreachable, graceful error shown - strength meter continues working	Must Have
FR-06.5	Full password or full SHA-1 hash shall never be transmitted externally	Must Have
FR-06.6	Breach result displayed alongside strength meter with red/green visual indicator	Should Have

2.7.7 Non-Functional Requirements

ID	Category	Requirement	Measure
NFR-01	Usability	New user can start and complete a game within 3 minutes without instructions	User testing
NFR-02	Security	bcrypt password hashing; ORM prevents SQL injection; API key server-side only	Code review
NFR-03	Performance	Page load < 3s; game interactions < 500ms; AI explanation < 5s; HIBP < 2s	Browser tools

ID	Category	Requirement	Measure
NFR-04	Compatibility	Functions on Chrome, Firefox, Edge - desktop and mobile browser	Manual test
NFR-05	Scalability	New game modules addable without modifying auth or scoring logic	Design review
NFR-06	Maintainability	PEP 8 compliance; code commented; README with setup and API key config	Code review
NFR-07	Reliability	No crash on invalid input or API failure; all inputs validated server-side	Testing
NFR-08	Accessibility	Sufficient colour contrast; instructions readable without prior security knowledge	Review
NFR-09	Privacy	No PII sent to Gemini or HIBP; AI explanations not persisted in database	Code review

2.7.8 System Constraints

Type	Constraint
Technical	Must be built using Python/Flask - no PHP, Node.js, or other stacks
Technical	SQLite for local dev; MySQL-compatible schema for optional deployment
Technical	Gemini API key stored as server-side environment variable - never client-side
Technical	HIBP integration must use k-Anonymity - full password never leaves server
Scope	Minimum 3, maximum 5 game modules within semester
Scope	No real phishing emails, malware, or live attack simulations
Budget	Zero monetary budget - all tools free or open-source
Data	No real PII beyond username/email; AI logs not persisted
Timeline	All core features testable by Week 14

Type	Constraint
Academic	All code original; external libraries documented and credited

2.7.9 Feasibility Study

A feasibility study determines whether the proposed project is achievable within the given constraints of time, resources, technology, and scope. This section evaluates the Chakravyuh - An Advanced Web Security Awareness Portal across five standard feasibility dimensions.

The goal of this study is to ensure the team commits to a project that is realistic, deliverable within a single semester, and genuinely useful to its intended audience.

2.7.9.1 Technical Feasibility

Technical feasibility examines whether the team has access to the right tools, platforms, and technical knowledge to successfully build the system.

Component	Chosen Technology	Justification
Backend Language	Python 3.x	Team familiarity; large support community; excellent web framework ecosystem
Web Framework	Flask	Lightweight, beginner-friendly, minimal boilerplate; ideal for semester scope
Frontend	HTML5, CSS3, JavaScript, Bootstrap 5	No additional learning curve; directly supports interactive game UI
Database	SQLite (dev) / MySQL (production)	SQLite requires zero setup for local demo; MySQL for optional deployment
ORM	SQLAlchemy	Abstracts database queries; reduces SQL injection risk; portable across DBs
Authentication	Flask-Login + bcrypt	Well-tested, standard session-based auth with hashed passwords
AI Integration	Google Gemini API (gemini-haiku)	Lightweight, cost-effective model for per-question AI explanations; API key stored server-side
Breach API	HaveIBeenPwned API v3	k-Anonymity model - only first 5 chars of SHA-1 hash sent; no full password transmitted

Component	Chosen Technology	Justification
Version Control	Git + GitHub	Industry standard; free for students; enables team collaboration
Hosting (Demo)	Local dev server / Render.com (free tier)	Sufficient for college demo; no cost involved

2.7.9.2 Operational Feasibility

Operational feasibility assesses whether the system will actually be used by its target audience and whether it fits within the operational environment of the college context.

- Primary: College students (1st–3rd year) with little to no cybersecurity background
- Secondary: Faculty members and internal guides reviewing the portal
- Admin: Project team members managing content via admin panel

Will Users Actually Use It?

Traditional text-based cybersecurity awareness content consistently fails to hold user attention. Research from cybersecurity training organisations shows:

- Game-based learning improves information retention by up to 40% compared to passive reading
- Interactive scenarios (e.g., phishing simulations) produce measurable behaviour change
- Leaderboards and badges significantly increase continued engagement in peer environments
- AI-generated, contextual explanations increase learner comprehension compared to static text feedback

The portal's design directly addresses these findings - interactive games, AI-powered explanations, breach checking, instant feedback, and gamification make it operationally compelling for the target demographic.

Requirement	Status
Internet browser access (users)	Available on all college lab computers and personal devices
Internet access for AI / API	Required for Gemini API and HaveIBeenPwned; core games function offline
Local server or hosting	Flask dev server sufficient for demo; free hosting for wider access
Admin to manage content	Admin panel built into the system; team manages content
User training required	Minimal - portal designed for intuitive, self-guided use
Maintenance post-submission	Optional; designed so new modules can be added without major rework

2.7.9.3 Economic Feasibility

Economic feasibility evaluates whether the project can be developed and deployed within an acceptable budget. Since this is a student mini-project, the primary constraint is zero direct monetary cost.

* The Google Gemini API free tier provides sufficient calls for demo-scale usage (a few hundred requests per day). If usage exceeds free limits during extended testing, API calls can be cached or rate-limited within the app.

The only non-monetary cost is team time - estimated at 8–12 hours per week per member across one semester, which is within the expected workload for a mini-project.

Item	Tool/Resource	Cost	Notes
Backend Framework	Python + Flask	Free	Open source
Frontend Libraries	Bootstrap 5, Chart.js	Free	CDN delivery
Database	SQLite / MySQL	Free	Bundled / free server edition
Development Tools	VS Code, Git, GitHub	Free	Free for students
UI Design	Figma / Canva	Free	Free tiers sufficient
Gemini API (AI)	Google Gemini API	Free*	Free tier sufficient for demo usage
HaveIBeenPwned API	HIBP API v3	Free	Public API; free for non-commercial use
Hosting (Demo)	Local server / Render.com	Free	Free tier for demo
Domain / SSL	Not required for submission	₹0	Local hosting only
Total Estimated Cost	-	₹0	Entirely open source / free-tier

2.7.9.4 Schedule Feasibility

Schedule feasibility confirms that the project can be completed within the allocated semester timeline. Below is a proposed high-level schedule.

Verdict: Schedule FEASIBLE. The 15–16 week timeline provides sufficient buffer. The modular approach allows the team to prioritise features and defer non-critical ones if time runs short.

2.7.9.5 Legal & Ethical Feasibility

This dimension confirms the project does not violate any legal, ethical, or institutional policies.

The portal educates users about real threats without enabling malicious behaviour - content is defensive only

No real phishing emails or malware are created; all scenarios are clearly simulated and labelled

AI explanations (via Gemini API) are used purely for educational feedback - no user personal data is sent to the API

HaveIBeenPwned checks use the k-Anonymity model - the full password is never transmitted over the network

User scores and progress data are stored only for the purpose of learning feedback

The project supports social good - it reduces real-world harm by improving awareness

Verdict: Legally and Ethically FEASIBLE. The project is educational in nature, both external APIs are used within their terms and with privacy-preserving methods, and user data is handled responsibly.

Area	Status
Software Licensing	All technologies used are open source (MIT, Apache, BSD) - no licensing conflicts
Gemini API Usage	Used under Google's API Terms of Service; non-commercial educational use; no user data sent to the API - only the question text and user's answer
HaveIBeenPwned Usage	Used under HIBP public API terms; k-Anonymity model ensures no full password or hash is transmitted; compliant with privacy by design principles
User Data Privacy	No real personal or financial data collected; password hashing with bcrypt; no third-party data sharing; AI interaction logs not persisted
Content Accuracy	Security content references established frameworks (OWASP, NIST) - factually grounded
Intellectual Property	All code written originally by team; no plagiarism; open source libraries credited
Institutional Policy	Project complies with college academic integrity and project submission guidelines

2.7.9.6 Feasibility Summary

Feasibility Dimension	Verdict	Key Justification
Technical	✔️ FEASIBLE	Python/Flask stack within team's skills; Gemini API and HIBP are lightweight and free within demo limits
Operational	✔️ FEASIBLE	Game-based + AI feedback format suits college students; self-guided and intuitive design
Economic	✔️ FEASIBLE	Zero monetary cost; all tools including Gemini API and HIBP available on free tiers
Schedule	✔️ FEASIBLE	Modular 16-week plan; API integrations scoped to specific phases; buffer time built in
Legal/Ethical	✔️ FEASIBLE	Educational use only; APIs used within ToS; k-Anonymity for HIBP; no personal data sent to Gemini
Overall Conclusion	✔️ PROCEED	All five dimensions indicate the project is viable and suitable for execution

2.7.10 Requirement Gathering & Analysis

This section documents the systematic process of identifying, gathering, and analysing the requirements for the Chakravyuh - An Advanced Web Security Awareness Portal. Requirements are the foundation of any successful software project - they define what the system must do, how well it must perform, and what constraints it must operate within.

2.7.10.1 Stakeholder Identification

Stakeholders are individuals or groups who have a direct interest in the project's outcome. Identifying them early ensures that requirements are gathered from all relevant perspectives.

Stakeholder	Role in Project	Key Interests
College Students	Primary end users of the portal	Engaging, easy-to-use games; meaningful progress feedback; AI-powered explanations
Internal Guide	Academic supervisor reviewing the project	Complete deliverable; adherence to project objectives; code quality
Project Team	Developers, designers, testers, content creators	Clear scope; manageable workload; learning opportunity

Stakeholder	Role in Project	Key Interests
College / Institution	Owner of the final project deliverable	Portfolio-worthy output; potential for future use/extension
Administrator (Team Member)	Manages portal content, users, and questions	Simple admin interface; ability to add/edit content without coding

2.7.10.2 Requirement Gathering Methods

The following methods were used to gather and validate project requirements. Given the academic context, the team relied on analysis rather than formal client interviews.

Method	Description	Outcome
Literature Review	Reviewed existing cybersecurity awareness platforms (Google Phishing Quiz, PhishSim, KnowBe4) and academic papers on game-based learning	Identified best-practice game mechanics and awareness content topics
Team Brainstorming	Structured discussion sessions among team members to define scope, modules, API integrations, and technical constraints	Agreed module list, API usage, role assignments, and initial feature priority
Peer Survey (Informal)	Informal questioning of 10–15 fellow students on their awareness of phishing, password hygiene, and social engineering	Confirmed awareness gap; validated relevance of chosen game topics
OWASP & NIST Framework Review	Consulted OWASP Top 10 and NIST cybersecurity guidelines for factually accurate content	Defined accurate, real-world content for games and quiz questions
Competitor Analysis	Reviewed 3–4 existing awareness portals to understand UI patterns, content structure, and feature sets	Identified gaps the proposed portal can fill
API Documentation Review	Reviewed Google Gemini API docs and HaveIBeenPwned API v3 docs to validate integration feasibility and privacy approach	Confirmed both integrations are feasible, free for demo scale, and privacy-safe

2.7.10.3 User Roles and Permissions

User roles and permissions are defined in Section 2.5 and are applied consistently throughout the requirements analysis.

2.7.10.4 Use Case Descriptions

Use cases describe the interactions between users and the system in a structured, scenario-based format. The most critical use cases are documented below.

Detailed use case descriptions are presented in Section 5.3 to keep the report structure non-repetitive.

2.7.10.5 Data Requirements

The following database entities are required to support all functional requirements. Relationships are described textually; a full ER diagram will be included in the Design Document.

The principal database entities are documented in Chapter 6, including the Entity Relationship Diagram, Entity Summary, and Data Dictionary.

2.7.10.6 Note on AI Interaction Log

The Gemini API is called per question/scenario to generate an explanation. By design, the system does not store AI interaction logs (prompts sent or responses received) in the database. This decision is intentional and based on three reasons:

- Privacy: The prompt includes the question and user's answer. Storing this per-user would create a detailed learning history that goes beyond what is necessary for the portal's purpose.
- Scope: For a single-semester mini-project, persisting AI logs adds schema complexity and storage cost with no direct benefit to the user experience.
- API Terms: Google's API usage policy does not require - and the team does not intend - to use interaction data for any purpose beyond generating the immediate explanation.

If a future version requires AI explanation caching for performance reasons, a separate AICache table (question_id, cached_explanation, generated_at, ttl) can be added without modifying the core schema.

2.7.10.7 Requirement Prioritisation (MoSCoW)

The MoSCoW method classifies all requirements into four priority levels to guide development order and scope management.

Must Have - Core Deliverables

- User registration, login, and logout with bcrypt password hashing
- Phishing Spotter game with at least 10 real-looking email samples
- Password Strength Lab with real-time strength indicator and explanation
- Password breach checker via HaveIBeenPwned k-Anonymity API (FR-06)
- AI-powered 'Explain This' feature via Gemini API after each question/scenario (FR-05)

- Graceful fallback for AI explanation when Gemini API is unavailable
- Quiz & Challenge module with at least 15 questions across difficulty levels
- Score recording and user dashboard displaying total points and module completion
- Admin panel for adding/editing quiz questions and phishing examples
- Responsive layout working on desktop browsers (Chrome minimum)

Should Have - High-Value Additions

- Social Engineering Simulator with at least 3 scenario branches
- Public leaderboard showing top 10 users by score
- Difficulty level selection per game module
- Progress bar / visual indicator showing modules completed
- Basic analytics page for admin (question pass/fail rates)
- 'AI Explanation' label badge clearly visible on AI-generated content
- Educational tooltip in Password Lab explaining what a data breach is

Could Have - Nice to Include

- Achievement badges for reaching score milestones
- Dark/light theme toggle for the UI
- AI explanation caching (AICache table) to reduce repeated API calls for the same question
- Mobile browser optimisation beyond basic responsiveness

Won't Have

- Mobile native app (iOS or Android)
- AI-based adaptive difficulty or personalised learning paths - NOTE: this is distinct from the scoped 'Explain This' feature (FR-05) which IS included. Full adaptive learning paths are out of scope.
- Multiplayer or real-time competitive mode
- Integration with college LMS or external authentication
- SSL certificate or production domain deployment
- Persistent AI interaction logs or conversation history across sessions

2.8 Timeline Chart

The project timeline below reconciles the feasibility note describing a 15–16 week window with the dated milestones in the Project Review Report 2026-27. The seven development phases are shown together with the review and approval milestones.

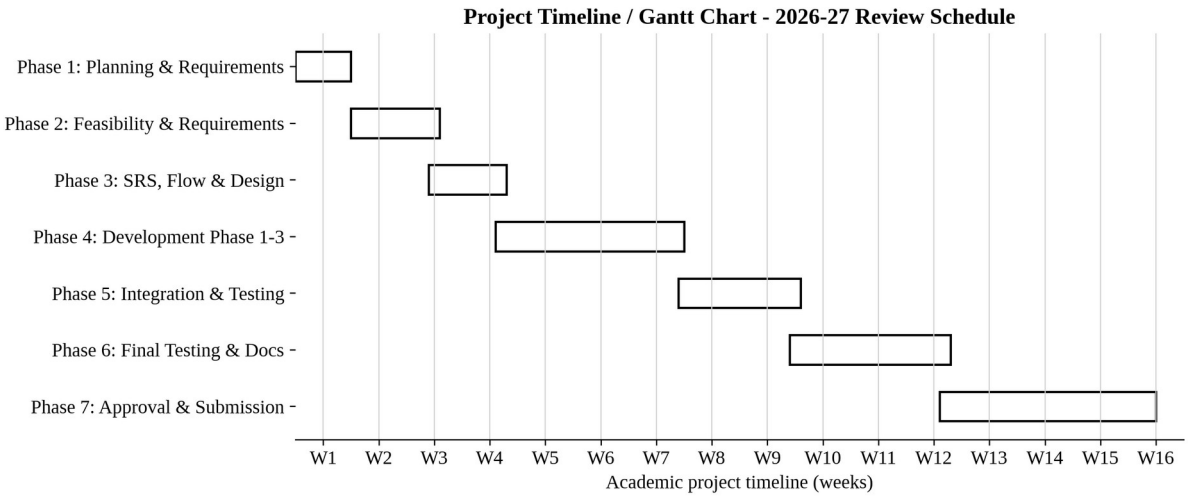


Figure 2.1 - Project Timeline / Gantt Chart (2026-27)

Milestone / Phase	Planned Date	Basis in Review Report
Research, title submission and group finalization	20-06-2026	Project Review Card
Presentation for definition finalization	01-07-2026	Project Review Card
Feasibility study, requirement gathering and existing-system research	17-07-2026	Project Review Card
SRS, Flow Chart and Timeline Chart	24-07-2026	Project Review Card
Use Case, DFD / OOAD, ER Diagram and Data Dictionary	24-07-2026	Project Review Card
Development Phase 1 - layout / form designing	07-08-2026	Project Review Card
Development Phase 2 / 3 - design with code and validation	28-08-2026	Project Review Card
Development Phase 4 - complete system and testing	04-09-2026	Project Review Card
Guide approval and document submission	30-09-2026	Project Review Card
HOD approval	01-10-2026 to 05-10-2026	Project Review Card
Dean approval	05-10-2026 to 07-10-2026	Project Review Card

3. System Flow Diagram

The following flowchart illustrates the complete user journey through the Chakravyuh - An Advanced Web Security Awareness Portal - from initial visit through login, module selection, gameplay, scoring, and AI-powered explanation.

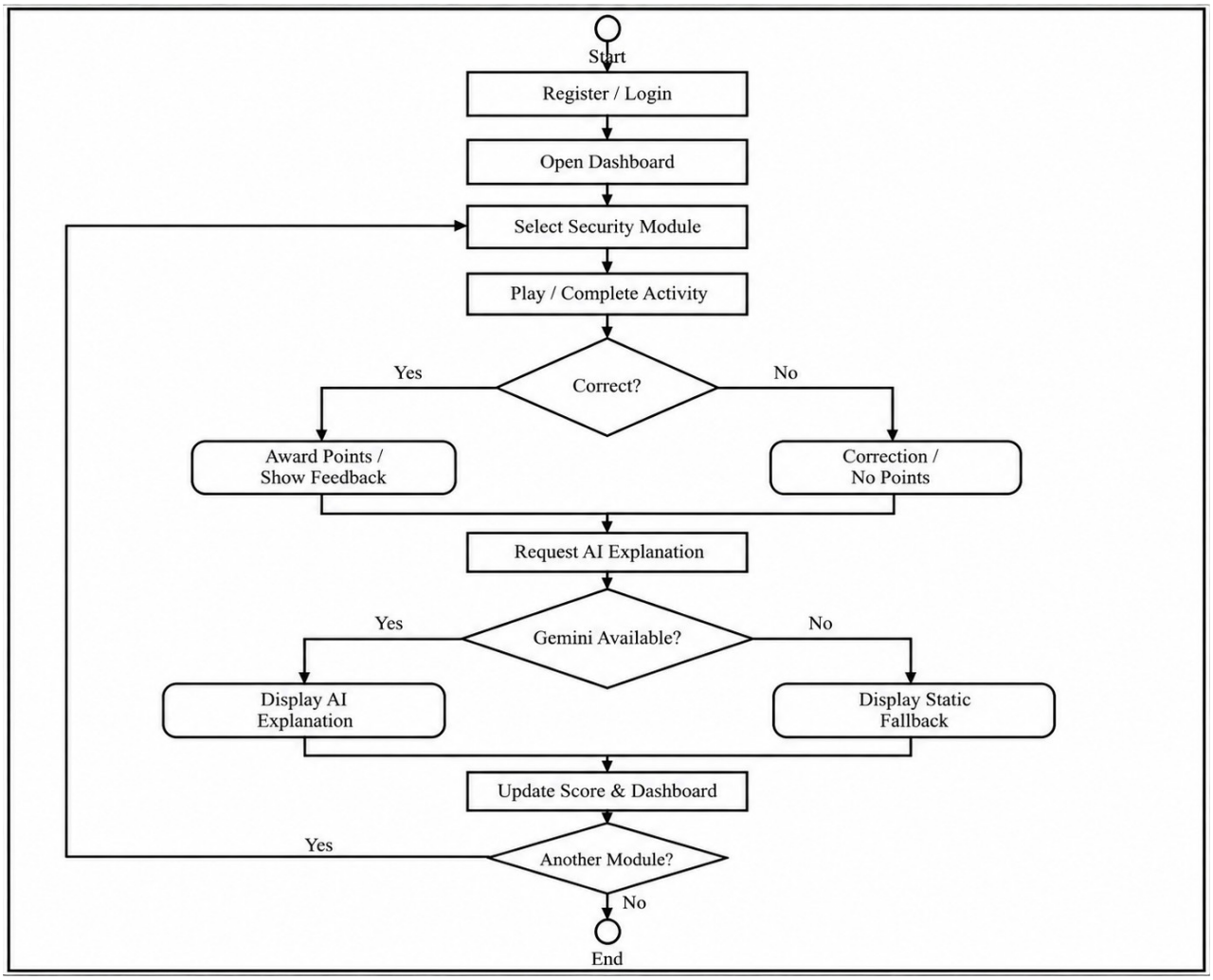


Figure 3.1 - Main System Flow Diagram

4. Data Flow Diagram (All Levels of DFDs)

Data Flow Diagrams model the movement of inputs, processing, storage, and outputs through the Chakravyuh - An Advanced Web Security Awareness Portal. The diagrams below follow the reference layout: external entities are rectangular, processes are circular, data stores use the two-line data-store symbol, and the major flows are horizontally aligned for readability.

4.1 Context Level DFD (Level 0)

The Context Level diagram represents the complete portal as a single process and shows its interactions with the Registered User, Admin, Gemini API, and HaveIBeenPwned API.

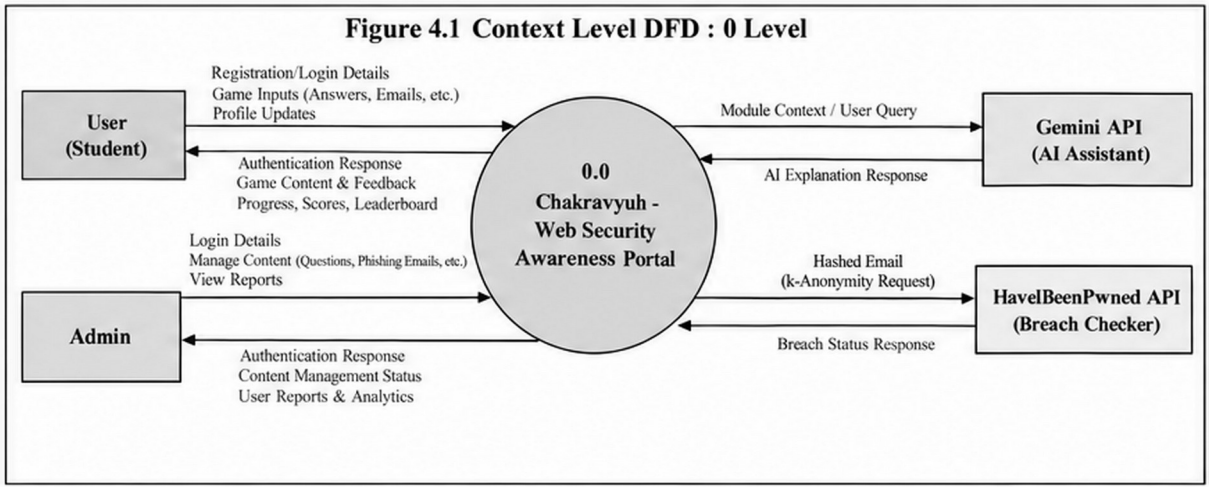


Figure 4.1 - Context Level DFD (Level 0)

4.2 Level 1 DFD - Registered User Side

The user-side Level 1 DFD expands the system into User Authentication, Game Engine, AI Explain, Scoring & Leaderboard, Dashboard, and Password Lab + HIBP, with the corresponding data stores and external services.

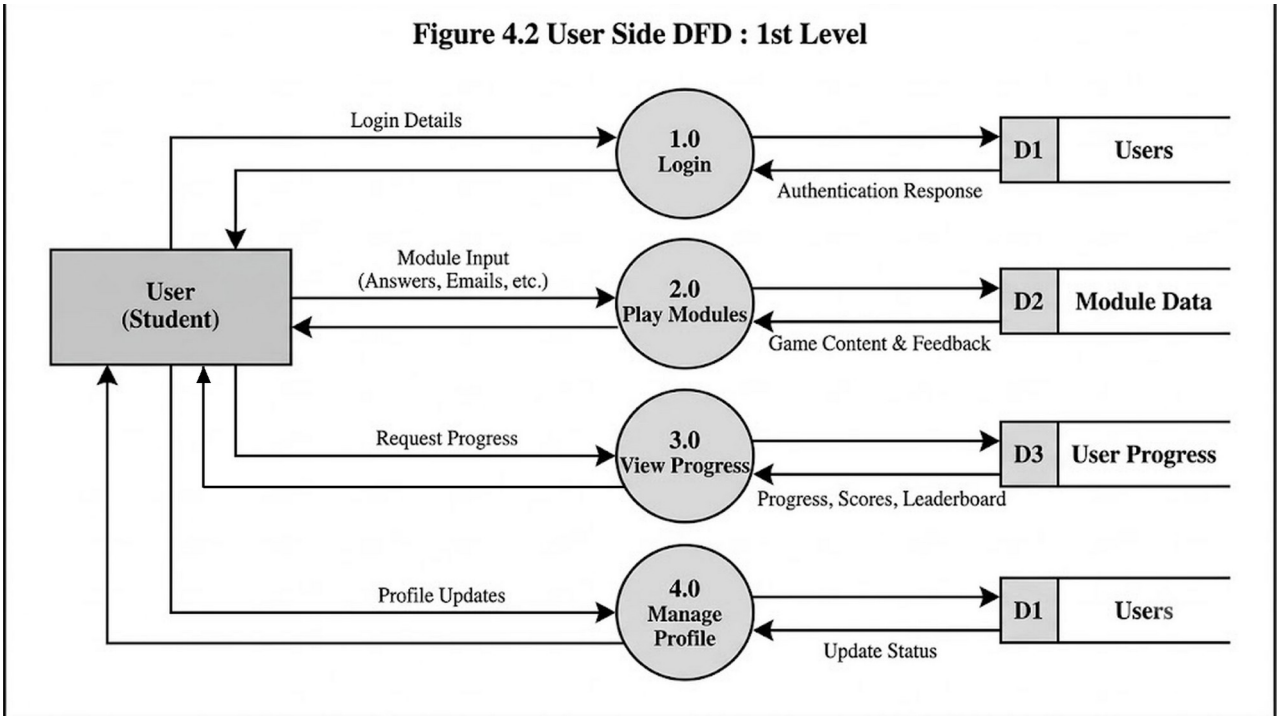


Figure 4.2 - Level 1 DFD: Registered User Side

4.3 Level 1 DFD - Admin Side

The admin-side Level 1 DFD expands the system into Admin Authentication, Content Manager, User Manager, and Analytics Engine, operating against the shared users, questions/examples, and scores/badges data stores.

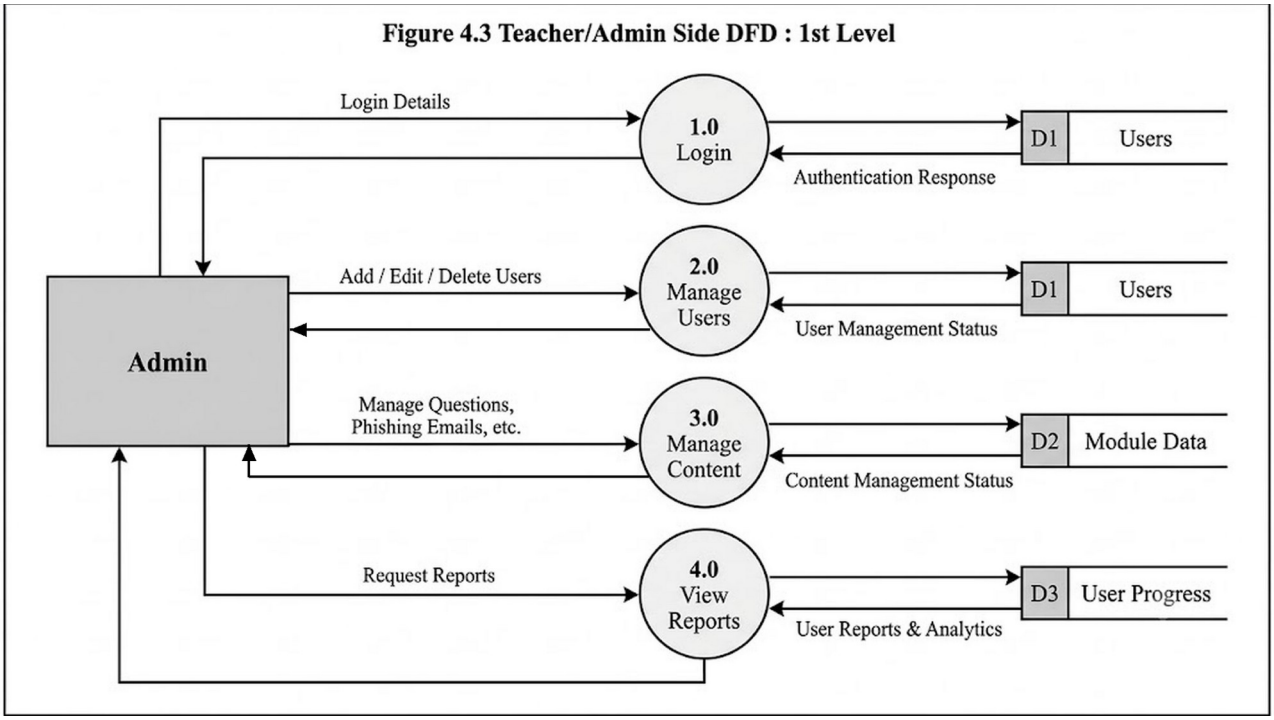


Figure 4.3 - Level 1 DFD: Admin Side

4.4 Level 2 DFDs

Level 2 DFDs further decompose the Game Engine and Admin Content Manager processes from the Level 1 diagrams into their principal sub-processes and data-store interactions.

4.4.1 Game Engine DFD

The Game Engine is decomposed into module loading, answer evaluation, explanation generation, and score recording.

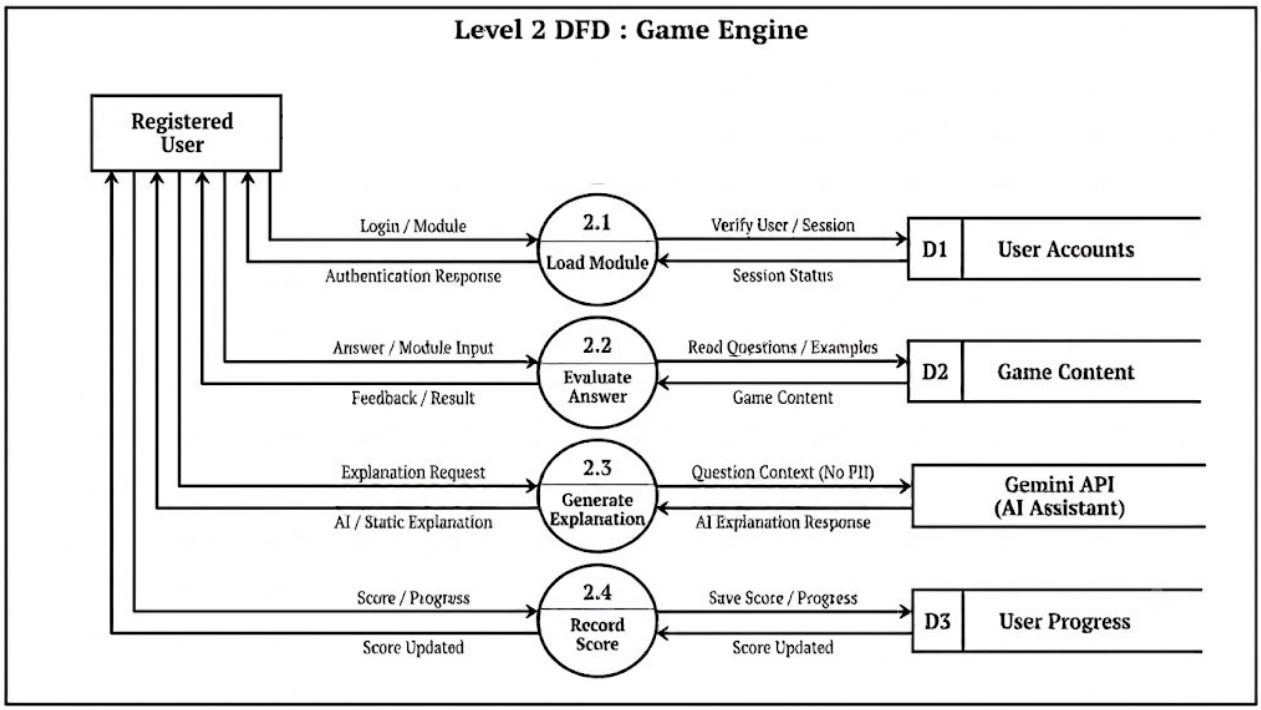


Figure 4.4 - Level 2 DFD: Game Engine

4.4.2 Admin Content Management DFD

The Admin Content Manager is decomposed into quiz management, phishing-example management, user analytics, validation, and persistence operations.

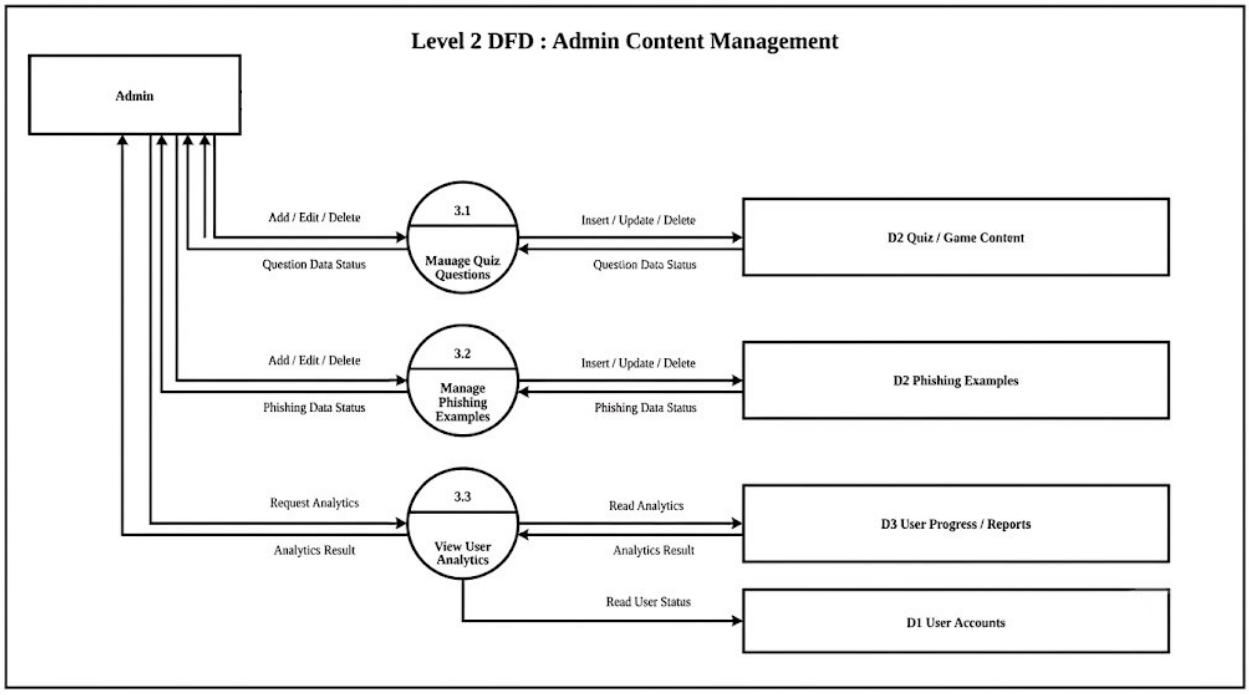


Figure 4.5 - Level 2 DFD: Admin Content Management

5. Use Case Diagram

Use Case Diagrams illustrate the interactions between system actors and the functional features of the Chakravyuh - An Advanced Web Security Awareness Portal. The principal actors are Registered User and Admin.

5.1 Use Case Diagram - Registered User

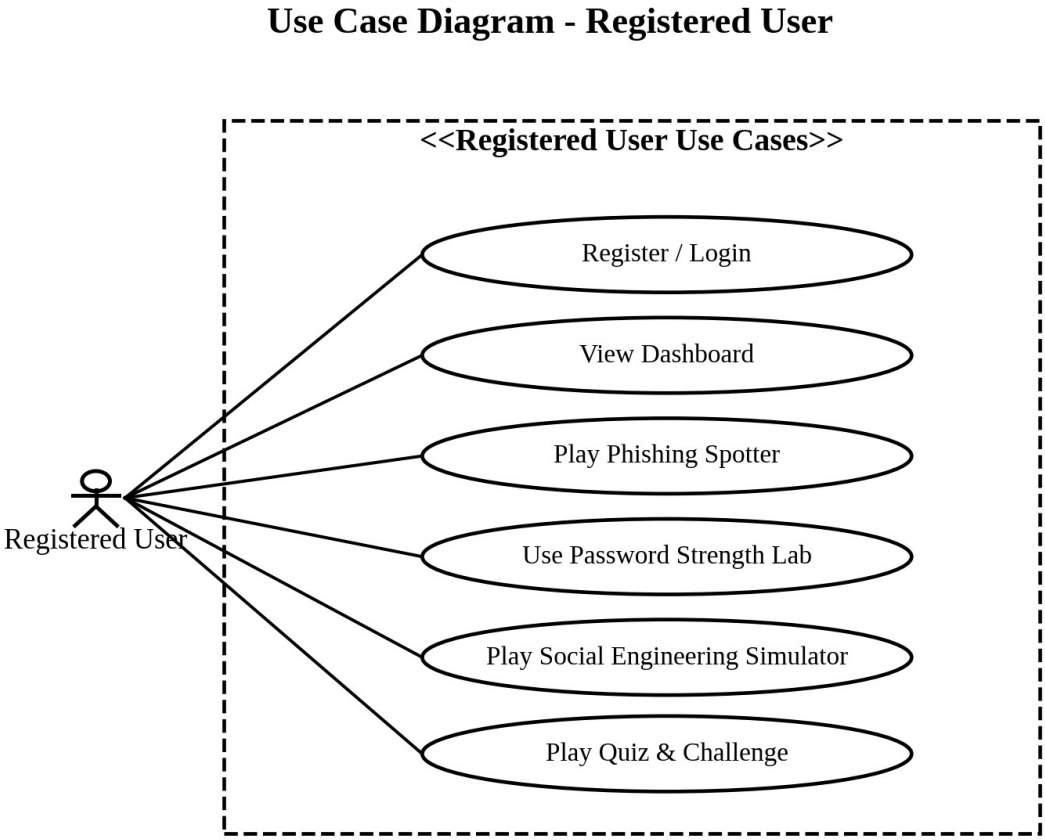


Figure 5.1 - Registered User Use Case Diagram

5.2 Use Case Diagram - Admin

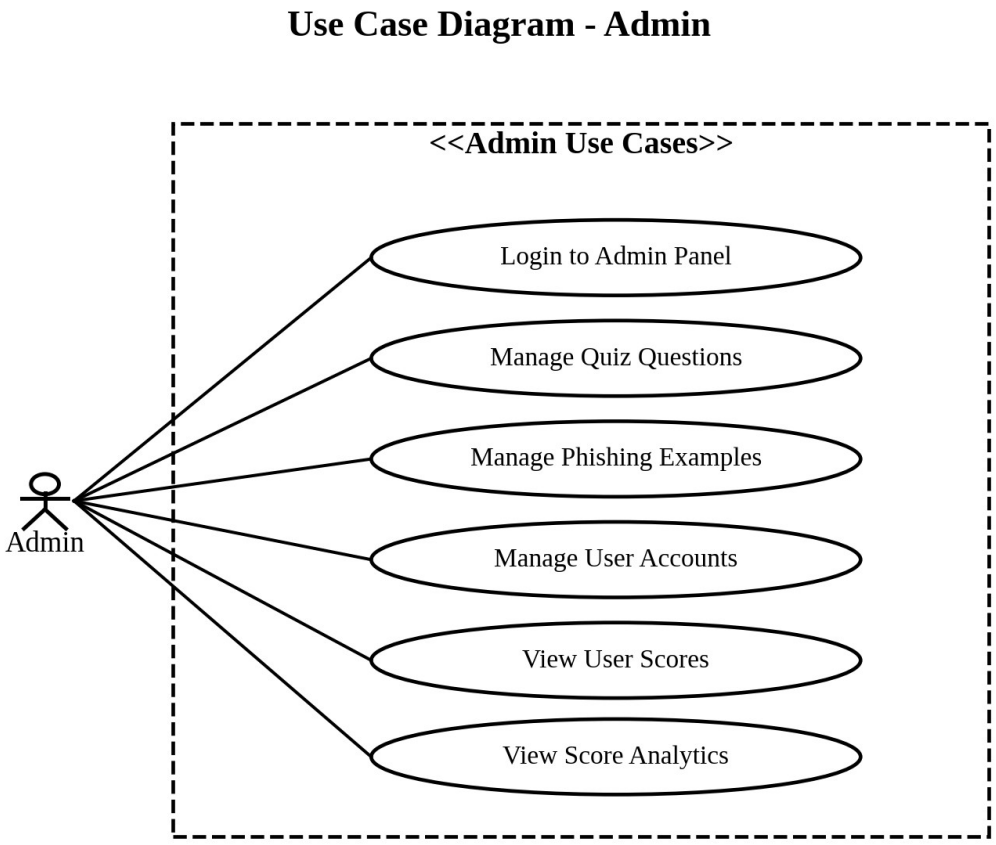


Figure 5.2 - Admin Use Case Diagram

5.3 Use Case Descriptions

The following use cases describe representative system interactions in a structured scenario format.

Use Case ID	**UC-01**
Use Case Name	User Registration
Primary Actor	Guest User
Trigger	User clicks 'Register' on the home page
Pre-condition	User is not logged in; registration form is accessible
Main Flow	1. User navigates to registration page 2. User enters username, email, and password 3. System validates inputs (unique email, password length ≥ 8 characters) 4. System hashes password using bcrypt 5. System creates user record in database 6. System redirects user to login page with success message

Use Case ID	UC-01
Alternative Flow	3a. Email already registered → System shows error: 'Email already in use' 3b. Password too short → System shows inline validation error 3c. Required field empty → System highlights missing field
Post-condition	New user account created; user can now log in

Use Case ID	UC-02
Use Case Name	Play Phishing Spotter Game with AI Explanation
Primary Actor	Registered User
Trigger	User selects 'Phishing Spotter' from the games menu
Pre-condition	User is logged in; phishing email examples exist in database
Main Flow	1. System loads a random phishing/legitimate email sample 2. User reads the email and selects 'Phishing' or 'Legitimate' 3. System evaluates answer and awards or withholds points 4. System sends question context (no PII) to Gemini API 5. AI-generated explanation is displayed with an 'AI Explanation' label 6. System records score to database 7. System offers 'Next Round' or 'Back to Dashboard'
Alternative Flow	4a. Gemini API times out → System displays pre-written static fallback explanation; game continues normally 3a. Wrong answer → No points awarded; explanation shown with correction
Post-condition	Score saved; user receives contextual AI explanation of the phishing indicator

Use Case ID	UC-03
Use Case Name	Use Password Lab with Breach Check
Primary Actor	Registered User
Trigger	User navigates to the Password Strength Lab
Pre-condition	User is logged in; HIBP API is reachable

Use Case ID	UC-03
Main Flow	1. User types a password into the lab input field 2. Client-side strength meter updates in real time 3. After 500ms debounce, system computes SHA-1 hash of password 4. System sends first 5 characters of SHA-1 hash to HIBP API 5. System compares returned suffix list locally 6. Breach status displayed: count of breaches found or 'Not found' 7. Educational tooltip visible explaining what a breach means
Alternative Flow	4a. HIBP API unreachable → System shows 'Breach check unavailable' message; strength meter still functions 1a. User clears the field → Breach indicator resets
Post-condition	User understands password strength and whether it appears in known breaches; no plaintext or full hash transmitted externally

Use Case ID	UC-04
Use Case Name	Admin Adds Quiz Question
Primary Actor	Admin User
Trigger	Admin navigates to Admin Panel → Quiz Content
Pre-condition	User is logged in with admin role
Main Flow	1. Admin clicks 'Add New Question' 2. Admin enters question text, four answer options, correct answer index, and difficulty level 3. Admin clicks Save 4. System validates all fields are filled 5. System saves question to database 6. System confirms with success notification
Alternative Flow	4a. Missing field → System highlights required fields with error message 6a. Database error → System shows error and retains form input
Post-condition	New question available in quiz pool immediately

5.4 Activity Diagrams

The activity diagrams model the main user learning flow and the administrative content-management workflow.

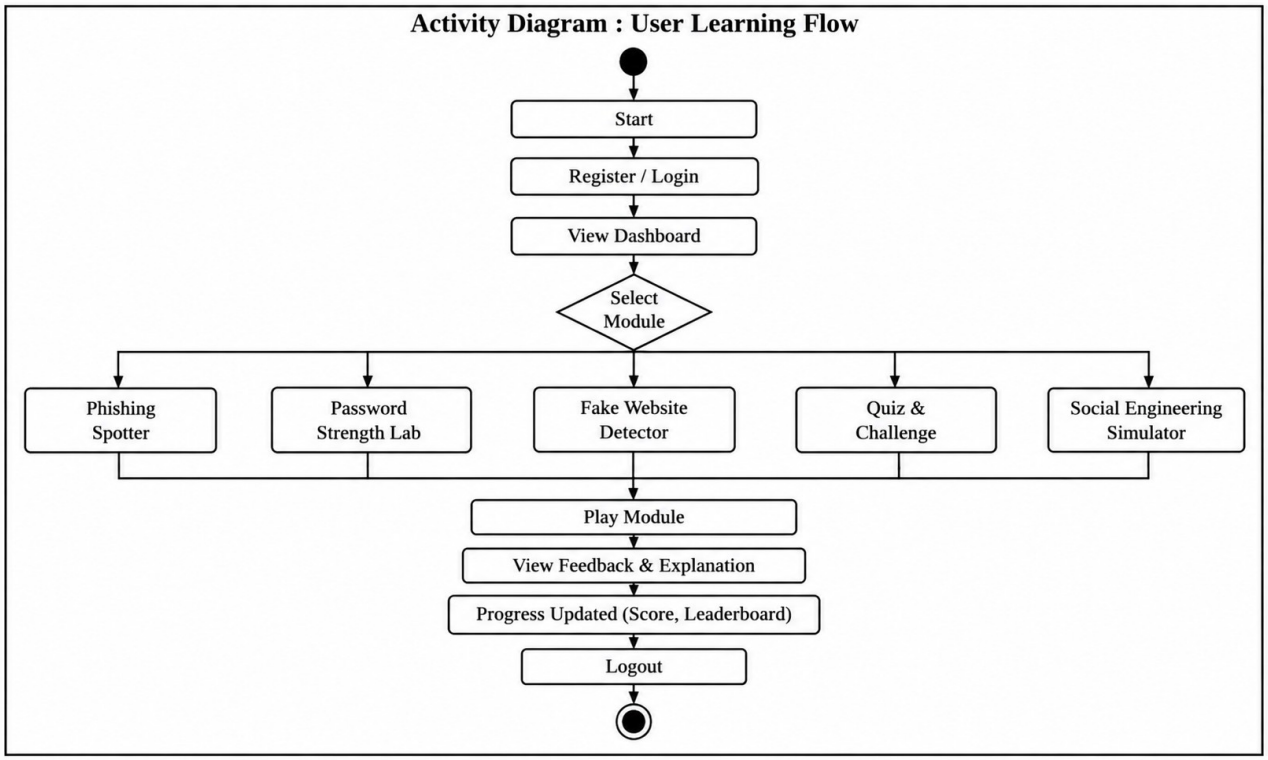


Figure 5.3 - User Activity Diagram

5.4.1 Admin Activity Diagram

The admin activity diagram represents authentication, content management, analytics, validation, and logout operations.

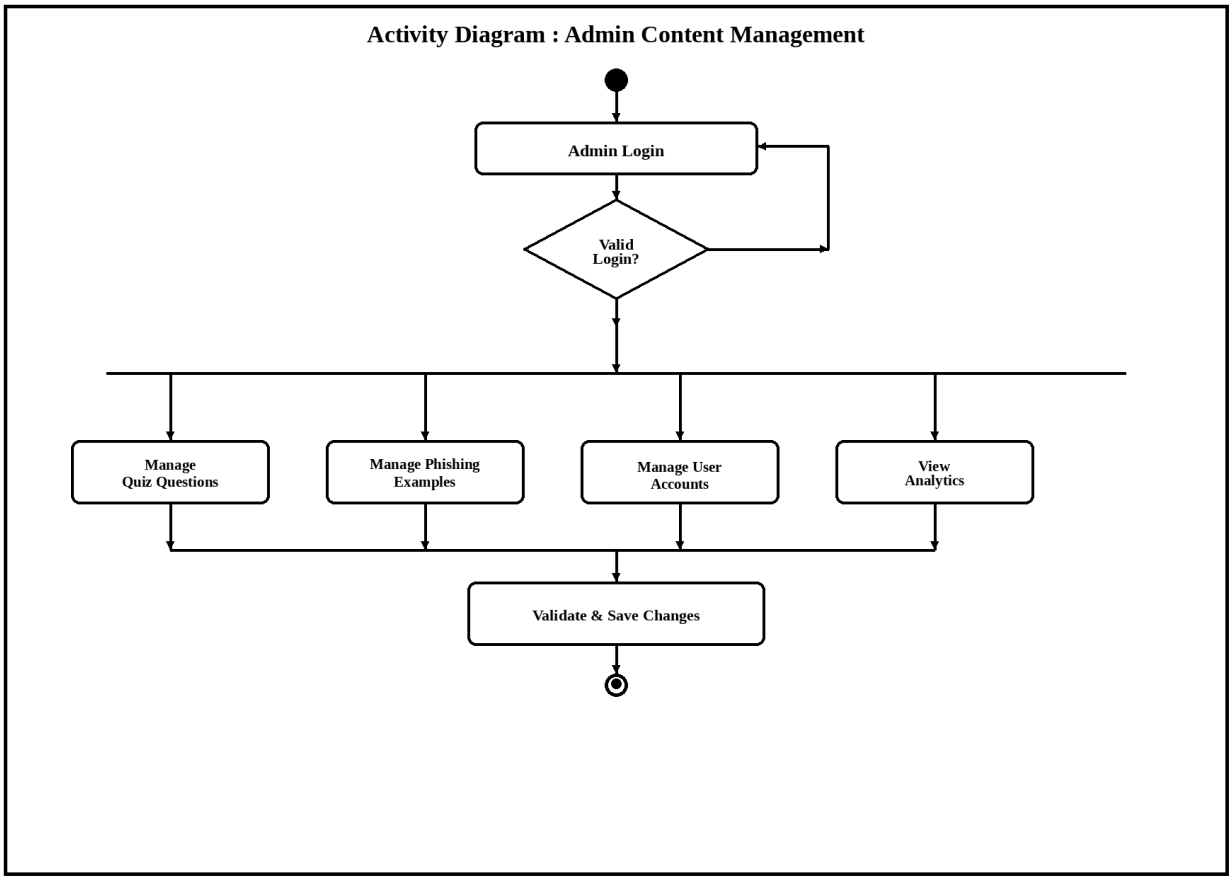


Figure 5.4 - Admin Activity Diagram

5.5 Class Diagram

The class diagram presents the principal application classes and their associations as an object-oriented view of the entities described in the SRS and Data Dictionary.

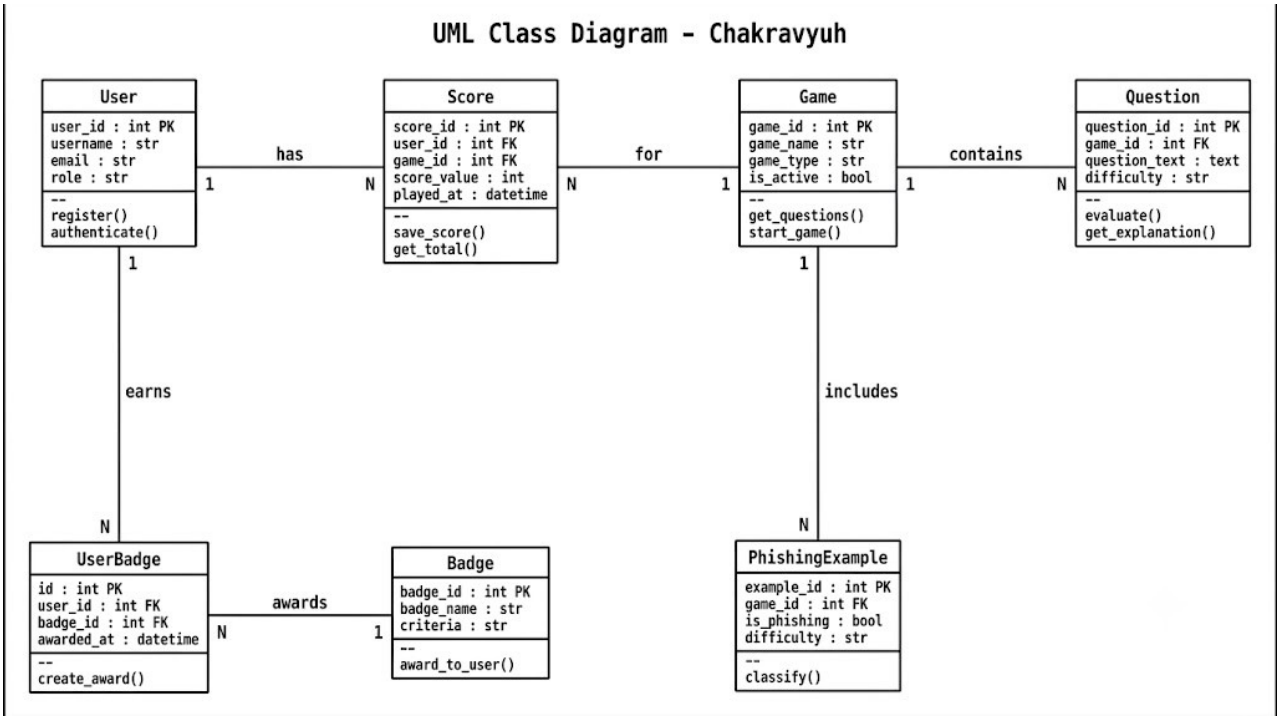


Figure 5.5 - UML Class Diagram

5.6 Sequence Diagram

The sequence diagram illustrates a representative game interaction in which the registered user submits an answer, the browser sends an HTTP request to the Flask backend, the database is accessed, and Gemini returns an explanation without receiving personally identifiable information.

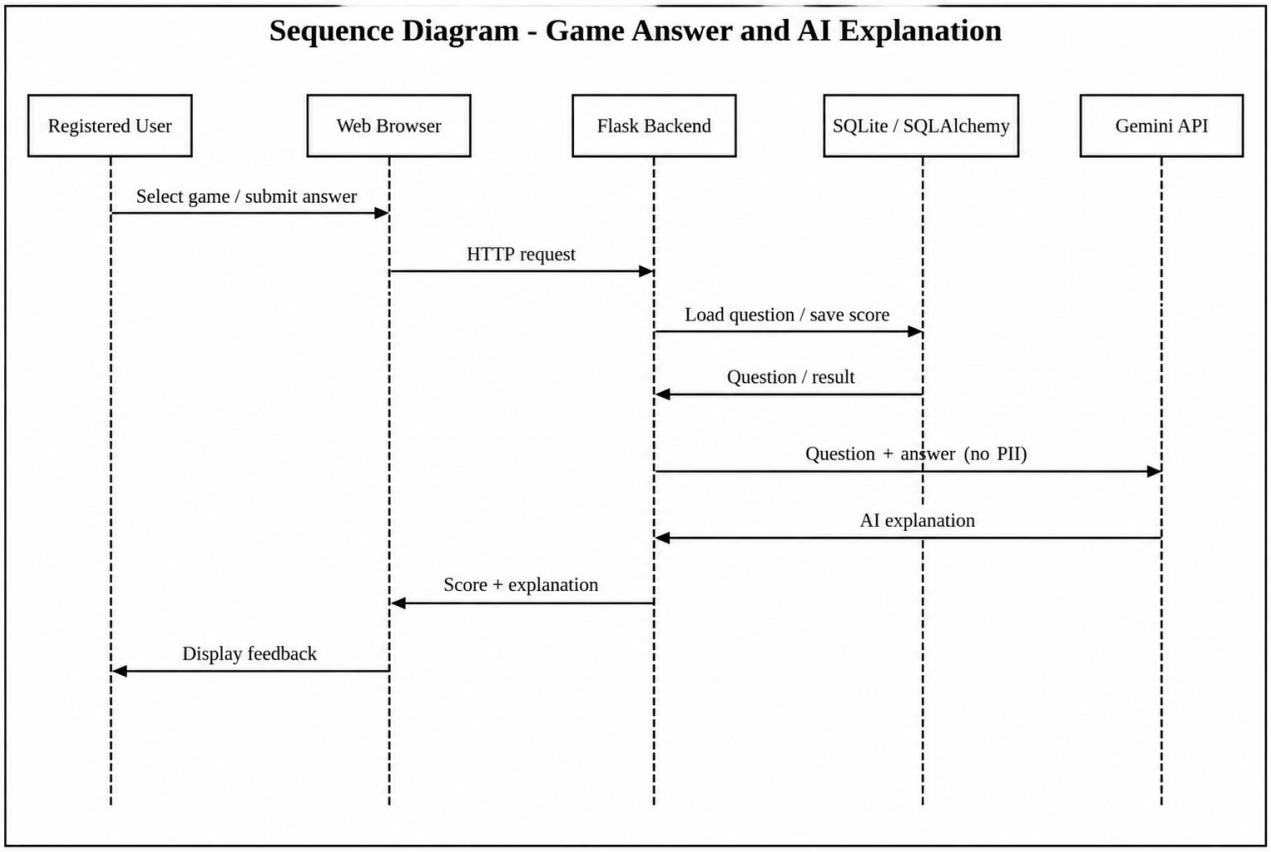


Figure 5.6 - Sequence Diagram: Game Answer and AI Explanation

6. Data Dictionary

6.1 Entity Relationship Diagram

The ER Diagram represents the database model of the Chakravyuh - An Advanced Web Security Awareness Portal. It uses Chen notation with rectangles for entities, diamonds for relationships, and ovals for key attributes. Primary keys are underlined, while the complete field definitions and foreign-key constraints are listed in the Data Dictionary.

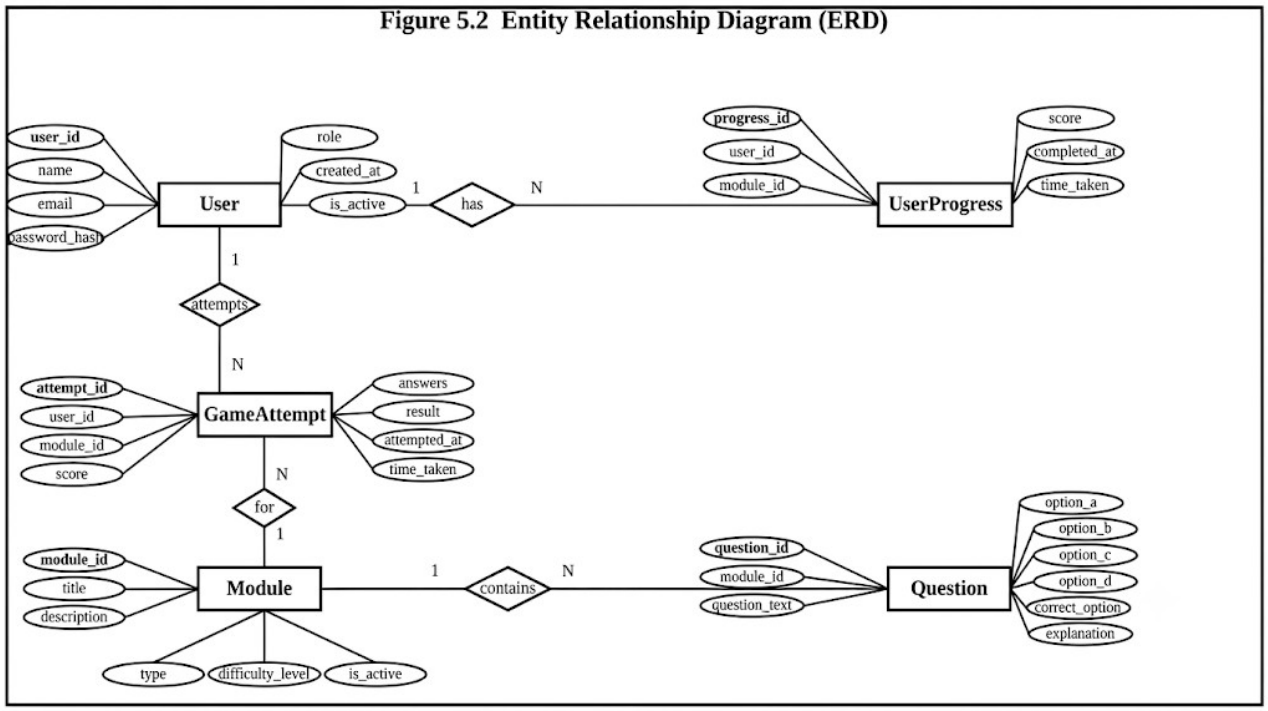


Figure 6.1 - Entity Relationship Diagram (Chen Notation)

6.2 Entity Summary

Entity	Key Attributes	Relationships
User	user_id (PK), username, email, password_hash, role, total_points, created_at	Has many: Score, Badge (via UserBadge)
Game	game_id (PK), game_name, game_type, description, is_active	Has many: Score, Question, PhishingExample
Question	question_id (PK), game_id (FK), question_text, options, correct_option, difficulty, static_explanation	Belongs to: Game
Score	score_id (PK), user_id (FK), game_id (FK), score_value, played_at	Belongs to: User, Game
Badge	badge_id (PK), badge_name, description, icon_url, criteria	Many-to-many with User via UserBadge
UserBadge	id (PK), user_id (FK), badge_id (FK), awarded_at	Junction: User ↔ Badge
PhishingExample	example_id (PK), game_id (FK), email_subject, sender, is_phishing, difficulty, static_explanation	Belongs to: Game

6.3 Data Dictionary

The Data Dictionary formally defines every field in every database table, including data types, constraints, and descriptions.

6.3.1 Table: users

Field	Type	Constraint	Description
user_id	INTEGER	PK, AUTO INCREMENT	Unique identifier for each user
username	VARCHAR(50)	NOT NULL, UNIQUE	Display name chosen during registration
email	VARCHAR(120)	NOT NULL, UNIQUE	User email address used for login
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash of user password - plaintext never stored
role	VARCHAR(10)	NOT NULL, DEFAULT "user"	Either "user" or "admin" - controls access level

Field	Type	Constraint	Description
total_points	INTEGER	DEFAULT 0	Cumulative score across all games played
created_at	DATETIME	NOT NULL	Timestamp of account creation

6.3.2 Table: games

Field	Type	Constraint	Description
game_id	INTEGER	PK, AUTO INCREMENT	Unique identifier for each game module
game_name	VARCHAR(100)	NOT NULL, UNIQUE	Display name (e.g., "Phishing Spotter")
game_type	VARCHAR(50)	NOT NULL	Category: phishing / password / social_eng / quiz
description	TEXT	NULLABLE	Brief description shown on module selection screen
is_active	BOOLEAN	DEFAULT TRUE	Allows admin to disable a module without deleting it

6.3.3 Table: questions

Field	Type	Constraint	Description
question_id	INTEGER	PK, AUTO INCREMENT	Unique identifier for each question
game_id	INTEGER	FK → games.game_id, NOT NULL	Which game module this question belongs to
question_text	TEXT	NOT NULL	The question shown to the user
option_a	VARCHAR(255)	NOT NULL	Answer option A
option_b	VARCHAR(255)	NOT NULL	Answer option B
option_c	VARCHAR(255)	NULLABLE	Answer option C (optional for some question types)
option_d	VARCHAR(255)	NULLABLE	Answer option D (optional for some question types)
correct_option	CHAR(1)	NOT NULL	Correct answer indicator: A, B, C, or D

Field	Type	Constraint	Description
difficulty	VARCHAR(15)	NOT NULL	One of: Beginner, Intermediate, Advanced
static_explanation	TEXT	NOT NULL	Fallback explanation shown if Gemini API fails

6.3.4 Table: scores

Field	Type	Constraint	Description
score_id	INTEGER	PK, AUTO INCREMENT	Unique identifier for each score record
user_id	INTEGER	FK → users.user_id, NOT NULL	The user who played this session
game_id	INTEGER	FK → games.game_id, NOT NULL	Which game was played
score_value	INTEGER	NOT NULL, DEFAULT 0	Points earned in this session
played_at	DATETIME	NOT NULL	Timestamp when the session was played

6.3.5 Table: badges

Field	Type	Constraint	Description
badge_id	INTEGER	PK, AUTO INCREMENT	Unique identifier for each badge
badge_name	VARCHAR(100)	NOT NULL, UNIQUE	Display name (e.g., "Phishing Pro")
description	VARCHAR(255)	NOT NULL	What this badge represents
icon_url	VARCHAR(255)	NULLABLE	Path to badge icon image file
criteria	VARCHAR(255)	NOT NULL	Condition to earn badge (e.g., "Complete Phishing Spotter")

6.3.6 Table: user_badges

Field	Type	Constraint	Description
id	INTEGER	PK, AUTO INCREMENT	Unique identifier for each award record

Field	Type	Constraint	Description
user_id	INTEGER	FK → users.user_id, NOT NULL	User who earned the badge
badge_id	INTEGER	FK → badges.badge_id, NOT NULL	Which badge was earned
awarded_at	DATETIME	NOT NULL	Timestamp when the badge was awarded

6.3.7 Table: phishing_examples

Field	Type	Constraint	Description
example_id	INTEGER	PK, AUTO INCREMENT	Unique identifier for each phishing example
game_id	INTEGER	FK → games.game_id, NOT NULL	Links to the Phishing Spotter game module
email_subject	VARCHAR(255)	NOT NULL	Subject line of the simulated email
email_body	TEXT	NOT NULL	Full body content of the simulated email
sender	VARCHAR(150)	NOT NULL	Simulated sender address (may look legitimate or spoofed)
is_phishing	BOOLEAN	NOT NULL	TRUE if this is a phishing email, FALSE if legitimate
static_explanation	TEXT	NOT NULL	Explanation of phishing indicators - fallback if Gemini API fails
difficulty	VARCHAR(15)	NOT NULL	One of: Beginner, Intermediate, Advanced

7. Screenshots of Development Phase - 1

7.1 System Architecture

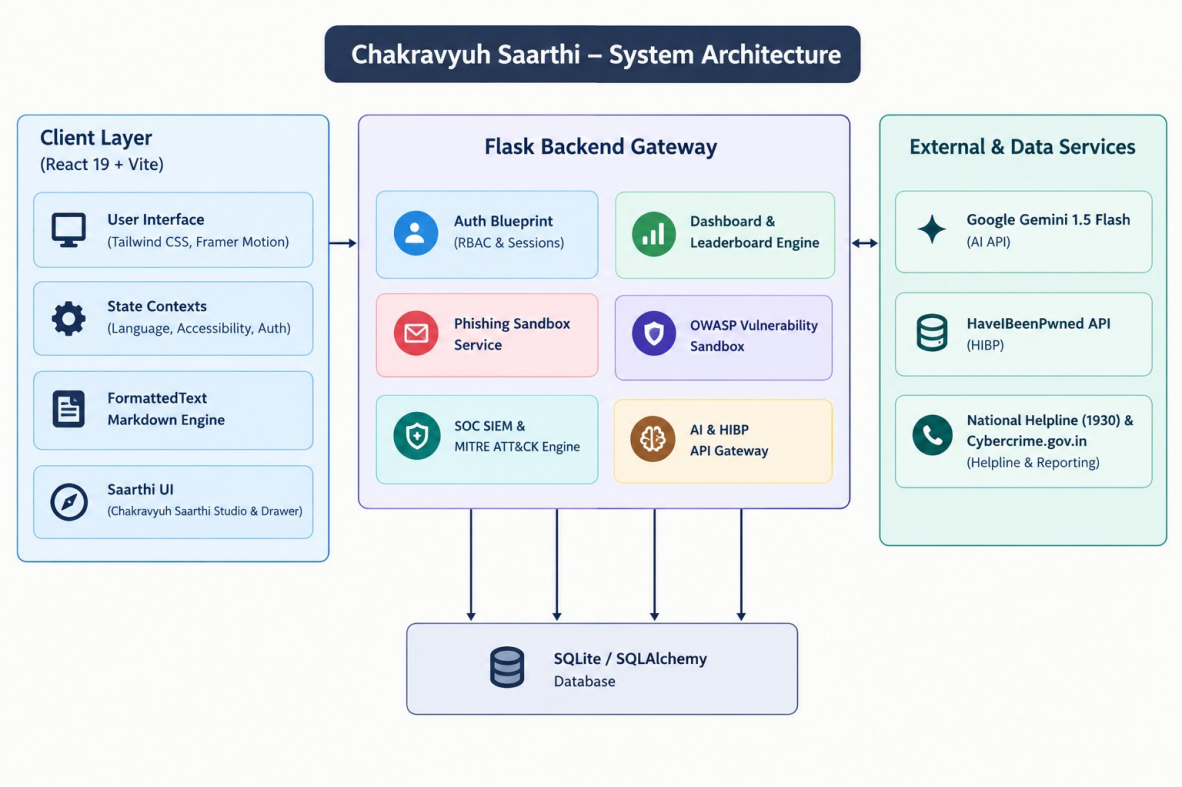


Figure 7.1 - System Architecture

7.2 User Registration / Login

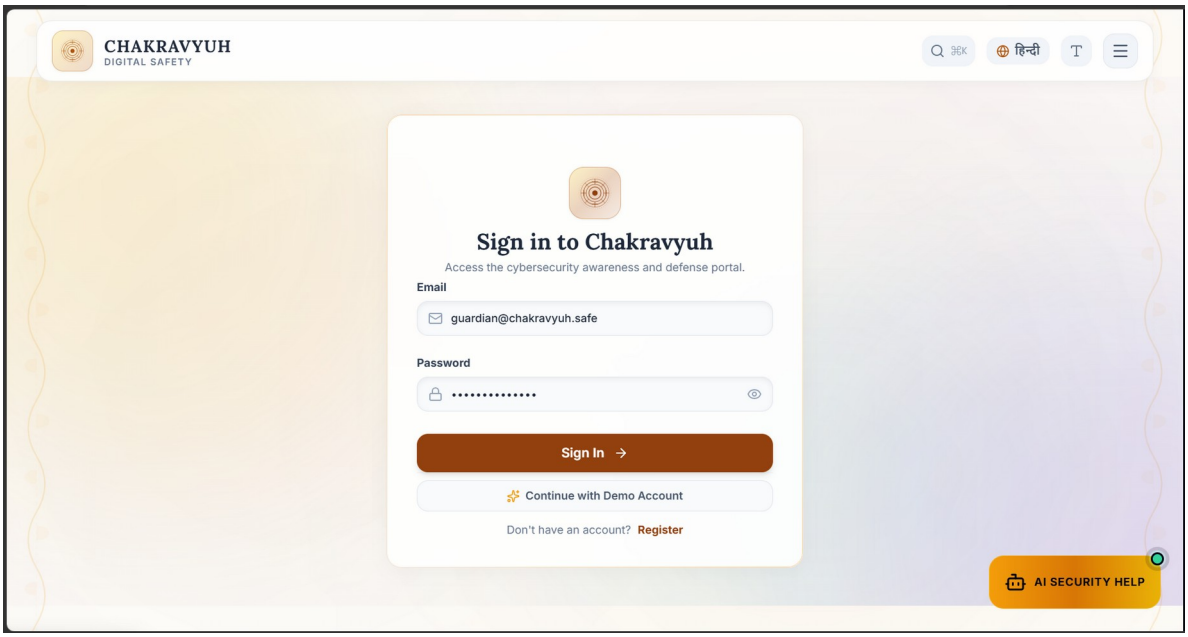


Figure 7.2 - User Registration / Login

7.3 Dashboard and Module Selection

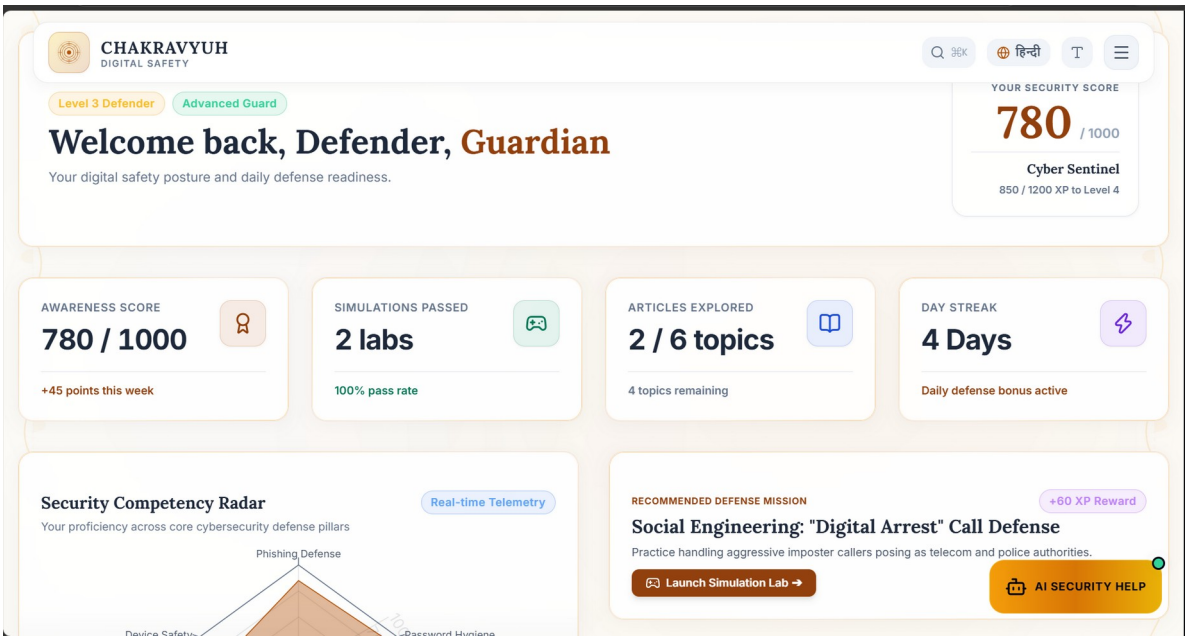


Figure 7.3 - Dashboard and Module Selection

7.4 Dashboard / Progress View

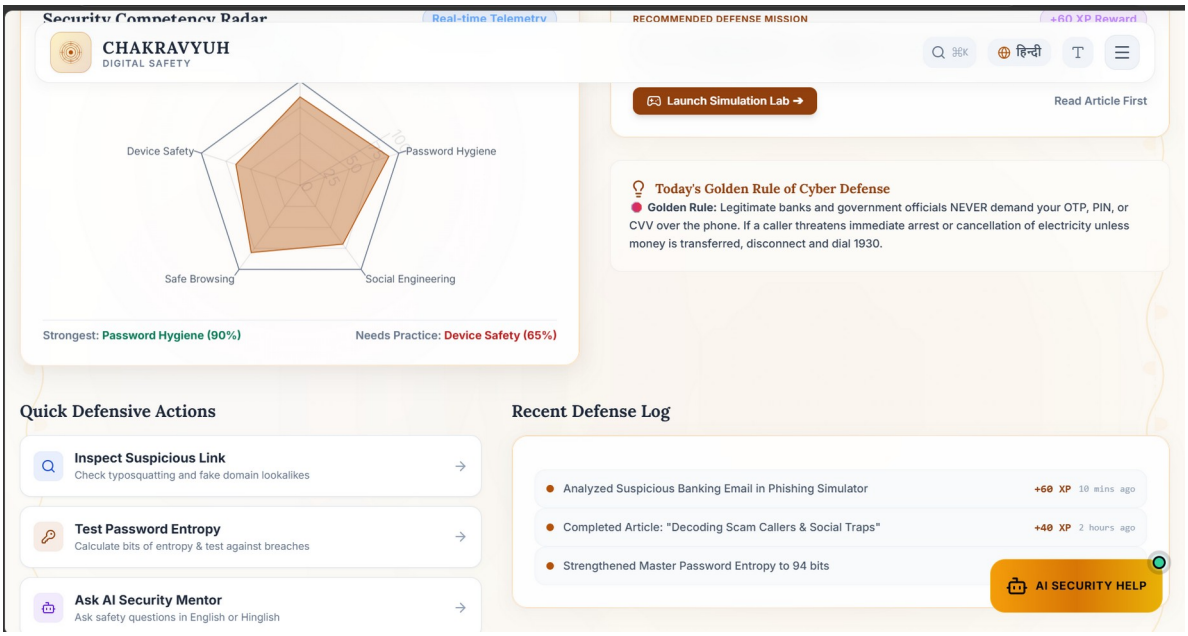


Figure 7.4 - Dashboard / Progress View

8. Screenshots of Development Phase - 2

8.1 Phishing Spotter / Password Lab

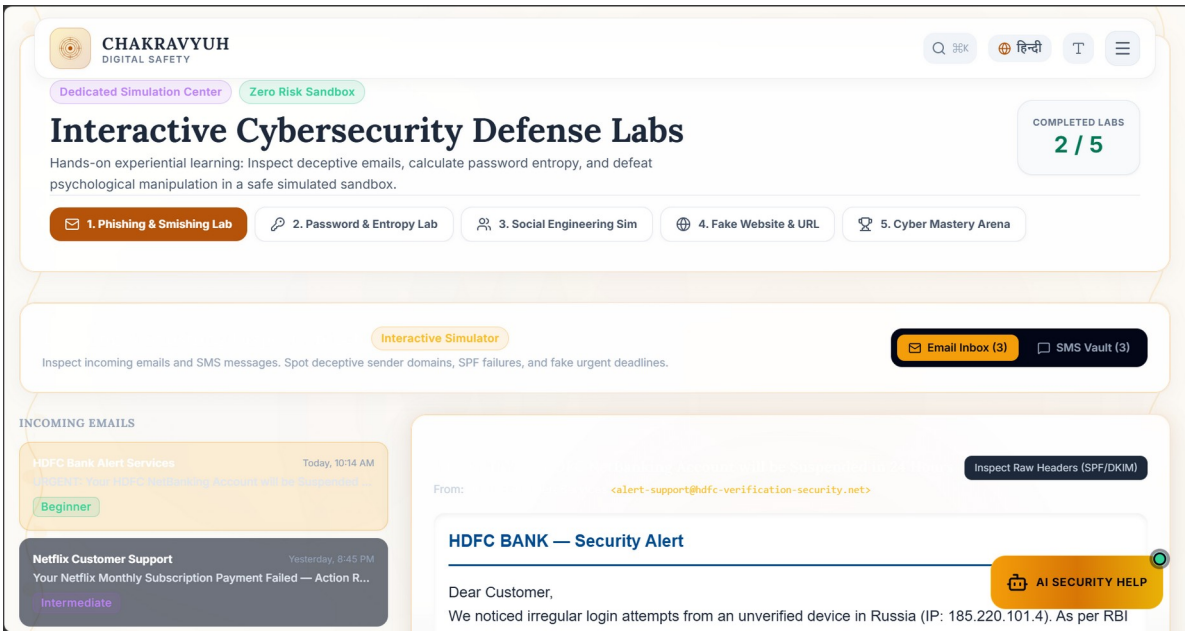


Figure 8.1 - Phishing Spotter / Password Lab

8.2 Quiz / Security Risk Activity

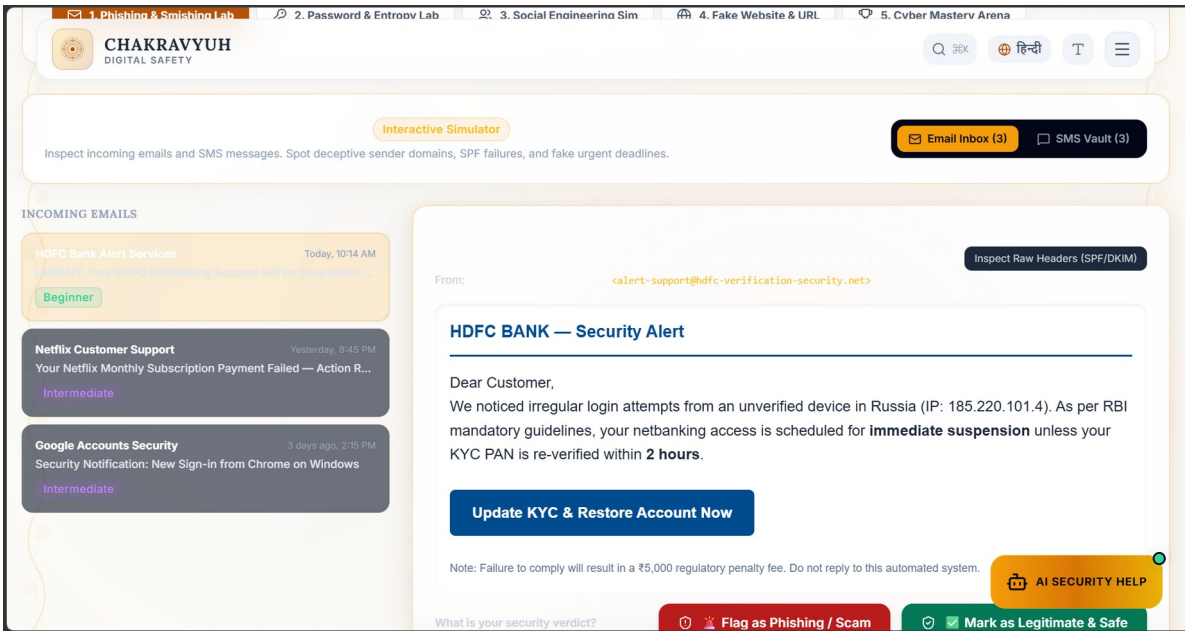


Figure 8.2 - Quiz / Security Risk Activity

8.3 Password Lab / Breach Check

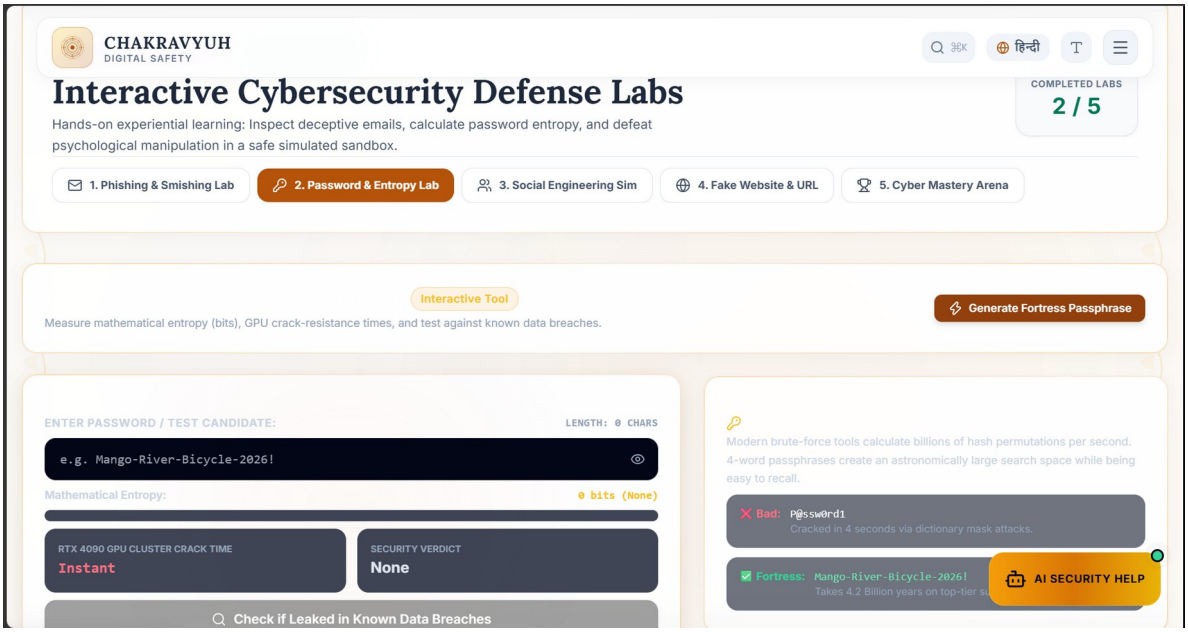


Figure 8.3 - Password Lab / Breach Check

8.4 Interactive Security Module

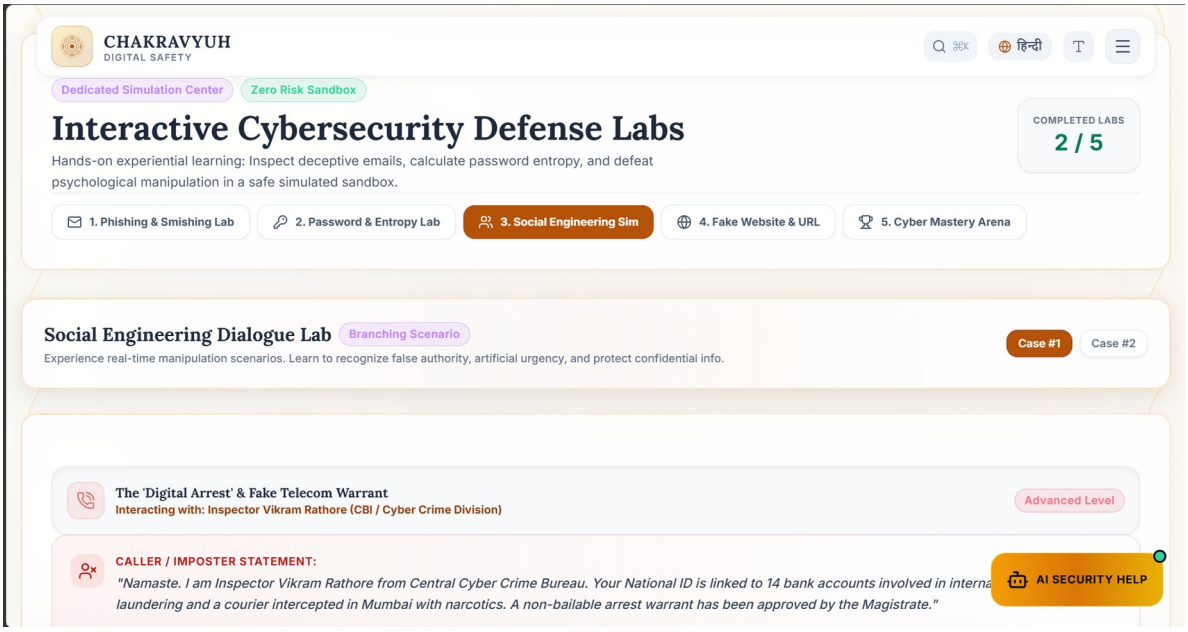


Figure 8.4 - Interactive Security Module

8.5 Interactive Quiz / Feedback

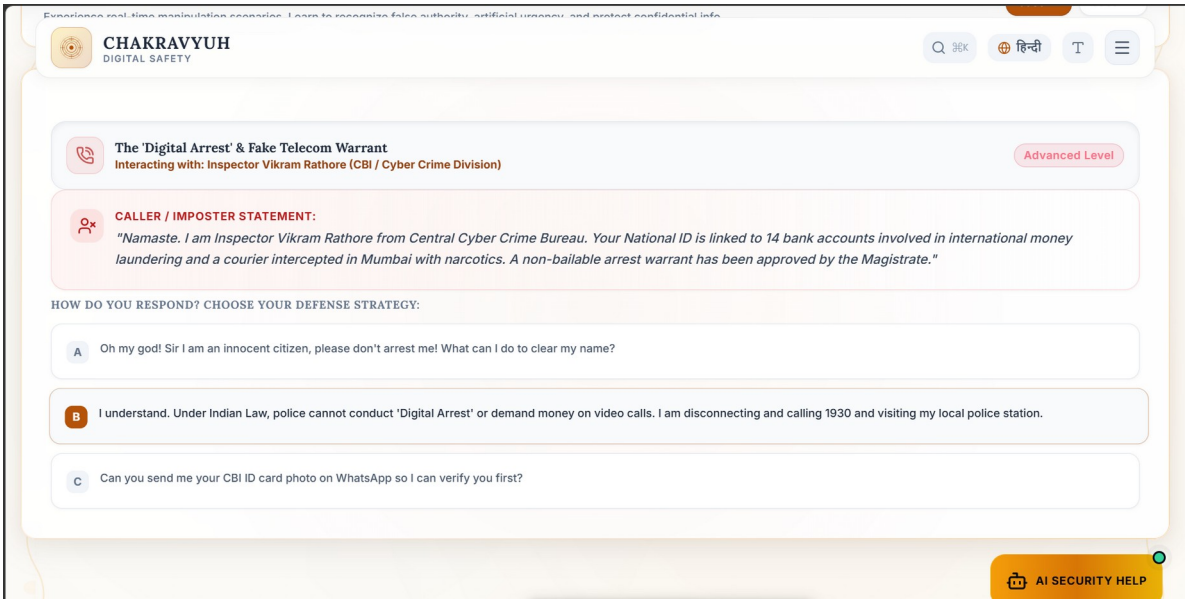


Figure 8.5 - Interactive Quiz / Feedback

8.6 AI Explanation / API Integration

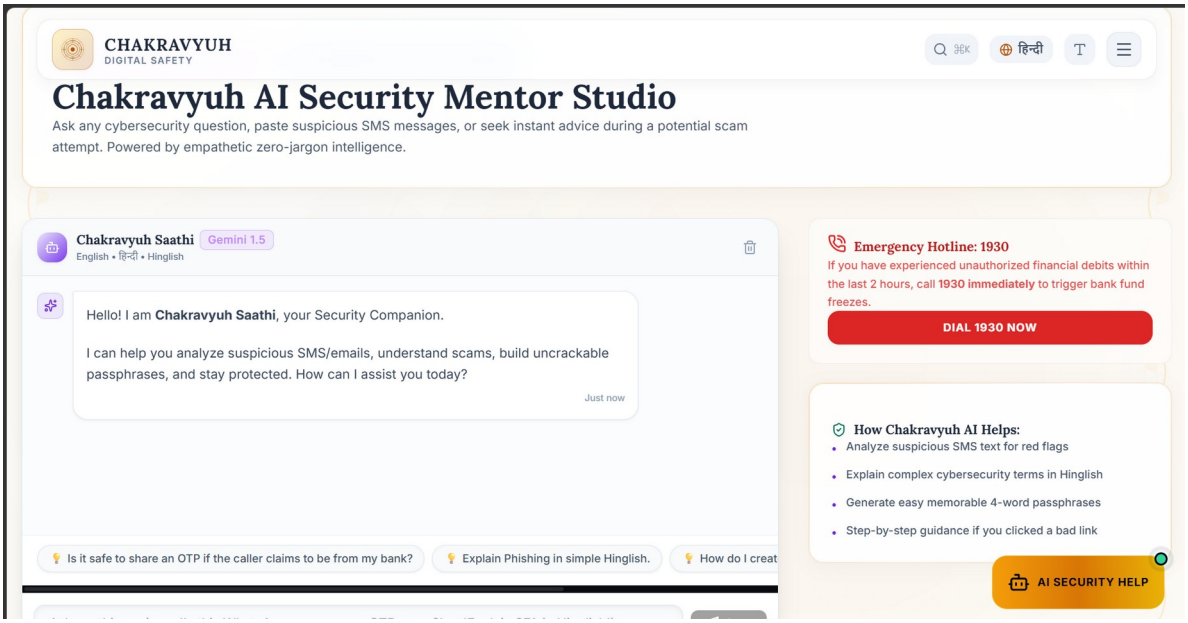


Figure 8.6 - AI Explanation / API Integration

9. Screenshots of Development Phase - 3

9.1 Progress / Leaderboard / Analytics

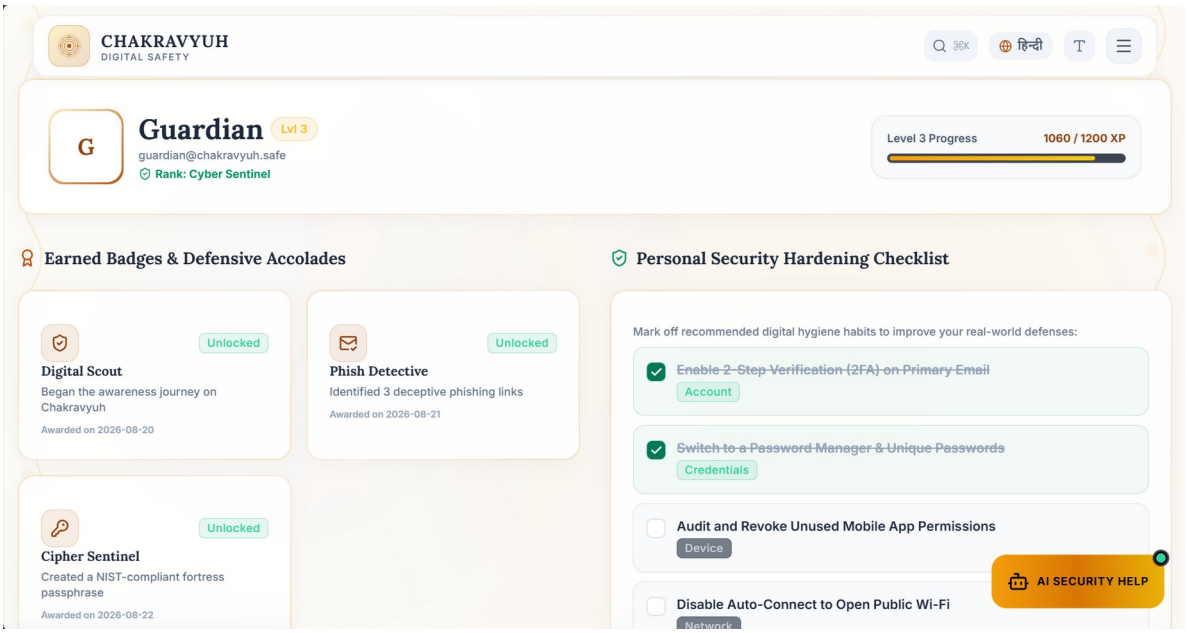


Figure 9.1 - Progress / Leaderboard / Analytics

9.2 Completion Certificate / Progress



Figure 9.2 - Completion Certificate / Progress

9.3 Analytics Dashboard



Figure 9.3 - Analytics Dashboard

9.4 Final Integrated Portal View

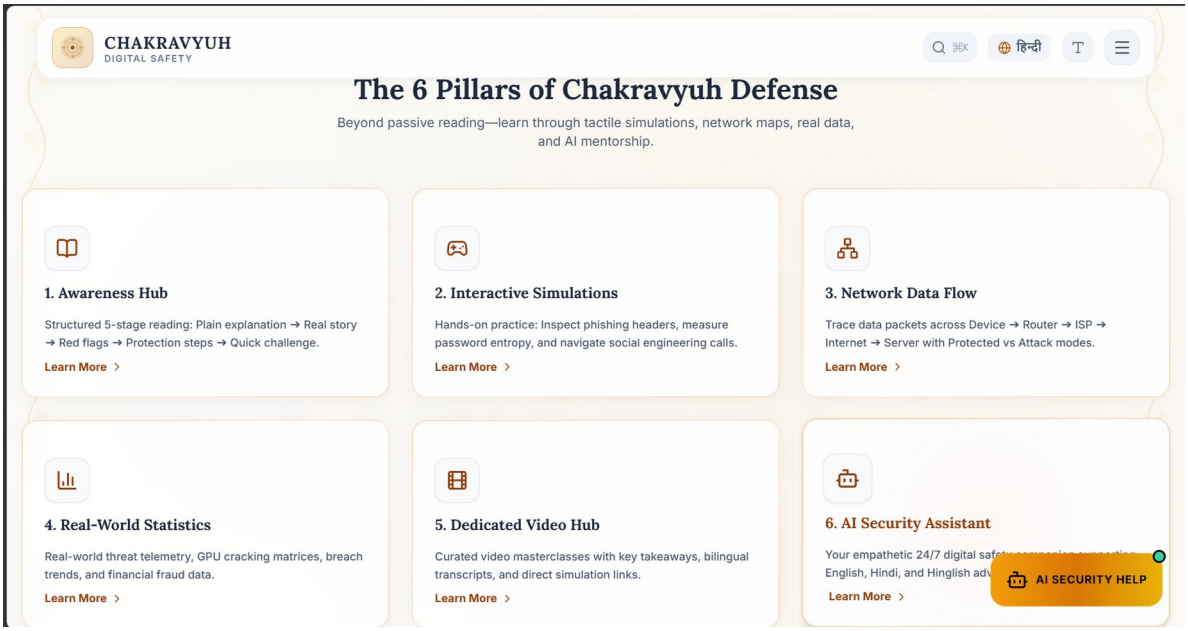


Figure 9.4 - Final Integrated Portal View

9.5 Threat Intelligence and Learning Content

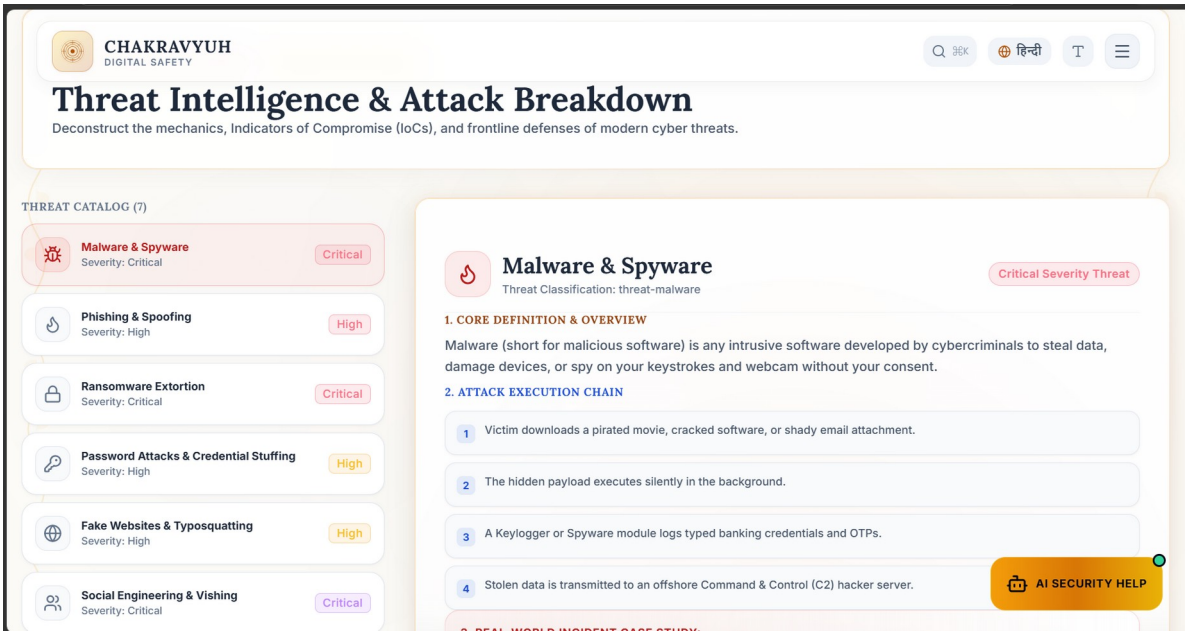


Figure 9.5 - Threat Intelligence and Learning Content

10. Conclusion

The existing cybersecurity awareness landscape can be broadly divided into two categories: free tools that are either too simple (Google Phishing Quiz, PhishTank) or too technical (OWASP WebGoat) to serve a general student audience, and paid enterprise platforms (KnowBe4, Cybrary) that are comprehensive but entirely inaccessible to students due to cost, complexity, and target audience mismatch.

The Chakravyuh - An Advanced Web Security Awareness Portal addresses a genuine gap in this landscape. It is the only solution in this comparison that is simultaneously free, beginner-friendly, gamified across multiple module types, customisable by educators, locally deployable, and specifically designed for college students. While it cannot match the content depth or professional credibility of enterprise platforms, it does not need to - its goal is to raise awareness and build practical recognition of everyday threats among a demographic that existing tools have largely ignored.

The comparative analysis confirms that the proposed system is not a duplication of existing work but a meaningful and targeted addition to the cybersecurity awareness ecosystem, positioned where the real awareness gap exists.

Considering the documented functional requirements, feasibility analysis, comparative study, and defined constraints, the Chakravyuh - An Advanced Web Security Awareness Portal provides a feasible semester-scale implementation with clear opportunities for later extension. The final system therefore meets the central objective of making cybersecurity awareness more interactive, approachable, and relevant to its intended student audience.

The project also remains honest about its scope. It is not intended to replace professional security-training platforms, certification programs, or production-scale phishing simulation systems. Its purpose is narrower and more practical: provide college students with an accessible environment in which common cybersecurity situations can be recognized, answered, explained, and revisited.

From an educational perspective, the proposed portal combines several elements that are normally distributed across different resources: practical threat-recognition exercises, password awareness, social-engineering scenarios, quiz-based learning, scoring, progress tracking, and an administrator-controlled content area. This combination supports repeated practice rather than one-time reading of security guidance.

10.1 Limitations

- The semester scope limits the initial content library and the depth of each module. The project comparison also notes that established commercial platforms have much larger libraries built over years, so the admin content-management capability is important for future expansion.
- The local/free-tier deployment model is suitable for a college demonstration and internal use, but it is not intended to represent a production deployment with high availability, dedicated infrastructure, or large-scale concurrent usage.
- The learning content concentrates on common, beginner-level threats. Advanced exploitation, live attack infrastructure, real phishing campaigns, malware samples, and production attack simulations remain outside the documented academic scope.
- The breach-check feature is designed around the public HaveIBeenPwned service and the information explicitly entered by the user. The system does not collect live external account credentials, and the SRS requires the k-Anonymity model so a full password or full SHA-1 hash is not sent externally.
- SQLite is appropriate for local development and the semester project, but its single-file design is not intended for a high-concurrency production environment. The feasibility and SRS documents identify a MySQL-compatible schema as an optional deployment direction if the system is later scaled.
- The project is developed and tested by a three-member student team within a single-semester schedule. Consequently, maintenance, security patching, content expansion, and long-term validation remain dependent on future project teams or institutional adoption.

11. Future Enhancement

The following enhancements are based on the project documentation and are intended to extend the portal beyond its current semester scope. They are future possibilities rather than features claimed as part of the current implementation.

- AI-based personalized difficulty (adaptive quizzes) - A future version could adjust question difficulty according to a learner's previous performance, while keeping the current scoped Explain This feature separate from full adaptive learning paths.
- VirusTotal URL/file scanner integration - A future module could allow users to examine suspicious URLs or files through an established scanning service, extending the awareness portal beyond the currently documented game scenarios.
- Live threat-intel map (AbuseIPDB + Leaflet.js) - Threat-intelligence data could be presented on an interactive map to help learners understand how malicious infrastructure and reported activity can be visualised.
- Shodan network exposure checker - A controlled educational integration could demonstrate how publicly visible network services and exposed assets can create security risk, subject to safe and ethical usage.
- NVD CVE feed & richer achievement badges - Future versions could use NVD vulnerability information for current security examples and expand the achievement system to provide more detailed recognition of learner progress.

These enhancements are intentionally presented as future scope. They extend the current portal toward broader threat intelligence, richer learner personalisation, and additional security-awareness exercises without changing the core semester-project objective.

12. References

[1] Bootstrap (2024). Bootstrap 5 - The World's Most Popular Frontend Toolkit. https://getbootstrap.com/

[2] Cisco / OpenDNS (2024). PhishTank - Community Anti-Phishing Site. https://www.phishtank.com

[3] Cybrary Inc. (2024). Cybrary - Cybersecurity Career Development Platform. https://www.cybrary.it

[4] Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From Game Design Elements to Gamefulness: Defining Gamification. Proceedings of the 15th International Academic MindTrek Conference.

[5] Flask Documentation (2024). Flask - A Python Microframework. https://flask.palletsprojects.com/

[6] Google (2024). Gemini API Documentation. https://ai.google.dev/gemini-api/docs

[7] Google / Jigsaw. (2019). Phishing Quiz. https://phishingquiz.withgoogle.com

[8] Hunt, T. (2024). HaveIBeenPwned Pwned Passwords API v3 - k-Anonymity Model. https://haveibeenpwned.com/API/v3#PwnedPasswords

[9] KnowBe4 Inc. (2024). KnowBe4 Security Awareness Training Platform. https://www.knowbe4.com

[10] NIST (2018). Framework for Improving Critical Infrastructure Cybersecurity, Version 1.1. National Institute of Standards and Technology.

[11] NIST. (2020). NIST Cybersecurity Framework. https://www.nist.gov/cyberframework

[12] OWASP Foundation. (2023). OWASP Top Ten Web Application Security Risks. https://owasp.org/www-project-top-ten/

[13] OWASP Foundation. (2024). WebGoat - A Deliberately Insecure Application. https://owasp.org/www-project-webgoat/

[14] PEP 8 - Style Guide for Python Code. https://peps.python.org/pep-0008/

[15] SQLAlchemy Documentation (2024). SQLAlchemy - The Database Toolkit for Python. https://docs.sqlalchemy.org/

[16] Verizon (2024). 2024 Data Breach Investigations Report. https://www.verizon.com/business/resources/reports/dbir/